Community Experience Distilled

Developing Mobile Games with Moai SDK

Learn the basics of Moai SDK through developing games

Francisco Tufró

[PACKT] PUBLISHING open source* community experience distilled

www.allitebooks.com

Developing Mobile Games with Moai SDK

Learn the basics of Moai SDK through developing games

Francisco Tufró

[PACKT] PUBLISHING
open source
community experience distilled

BIRMINGHAM - MUMBAI

www.allitebooks.com

Developing Mobile Games with Moai SDK

Copyright © 2013 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author, nor Packt Publishing, and its dealers and distributors will be held liable for any damages caused or alleged to be caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

First published: March 2013

Production Reference: 1200313

Published by Packt Publishing Ltd.
Livery Place
35 Livery Street
Birmingham B3 2PB, UK.

ISBN 978-1-78216-506-4

www.packtpub.com

Cover Image by Aldana Belen Gonzalez (plushcuanperfecto@gmail.com)

www.allitebooks.com

Credits

Author
Francisco Tufró

Reviewers
Eric Edelman
Marcos "marquete" Navarro
Adam Villalobos
Alejandro Jimenez Vilarroya

Acquisition Editor
Usha Iyer

Commissioning Editor
Ameya Sawant

Technical Editors
Saijul Shah
Dennis John

Copy Editors
Laxmi Subramanian
Brandt D'Mello
Insiya Morbiwala
Ruta Waghmare

Project Coordinator
Sherin Padayatty

Proofreader
Stephen Silk

Indexer
Monica Ajmera Mehta
Tejal Soni

Graphics
Aditi Gajjar

Production Coordinator
Pooja Chiplunkar
Nilesh Mohite

Cover Work
Pooja Chiplunkar

www.allitebooks.com

About the Author

Francisco Tufró has been captivated by computers and the possibility of using them to create new worlds since he was six years old. At age 14, he sort of hacked into a chat demo in Visual Basic and transformed it into a full-feature RPG chat, with support for maps, character sheets, and dice rolls. The years went by and he learned many things along the way, as any curious person does. Suddenly he found himself collaborating in various open-source projects, including Musix (a Linux distribution for musicians) and CLAM (working on the project for Google's *Summer of Code 2008*). He co-founded quov.is and worked as a Ruby on Rails developer for about 5 years while never forgetting about what drove him to computers in the first place, games.

He put together a team and created *The Insulines*, an old-school graphic adventure about rock 'n' roll and diabetes. It was thanks to this game that he first came into contact with Moai SDK. It took about 8 months of development. He fell so deeply in love with Moai SDK that now he's working full-time on it with Zipline Games.

He likes to call himself a developer, noting the difference from a programmer who is a person that has broad knowledge (not only in programming, but also in art, music, and other disciplines), perhaps not as deep as a specialized person does, but enough to tackle and solve problems in their entirety.

I want to thank Aldana for being the flower in concrete, my family for letting me break thousands of computers so that I could learn, the guys at Zipline Games for creating and (especially) sharing this amazing piece of software, and last but not the least, the guys at quov.is, with whom I have shared the most amazing experiences. I also want to thank the team behind *The Insulines* for a rushed but nice job.

---

www.allitebooks.com

About the Reviewers

Eric Edelman grew up in Indiana then moved out to the Seattle area to attend the DigiPen Institute of Technology. There he graduated with a Bachelor of Science degree in Real-Time Interactive Simulation while getting a double minor in Math and Physics. After that he worked at Microsoft writing automated tests for the XNA framework before moving to his position at Zipline Games. At Zipline Games, he helps to maintain Moai SDK as well as in creating in-house mobile games using Moai. He has worked on hit games such as *Wolf Toss* and *Slots Tycoon*—together getting more than 2.5 million players.

Marcos "marquete" Navarro (pronounced mark-ET-eh) grew up in a small town and figured out he wanted to become a computer programmer at the age of 13. Five years later, he moved to Buenos Aires to study Computer Science. He has had the fortune of meeting some amazing people while away from his hometown, a subset of which he now works with.

Although software development pays his bills, he's always had a thing for language, and he really enjoys pointing out people's mistakes to them.

He also likes to write about himself in the third person whenever the opportunity arises.

I want to thank my girlfriend Mercedes (my favorite linguist) for helping me throughout the review process and being supportive in general.

www.allitebooks.com

Adam Villalobos went to DigiPen Institute of Technology, and received a Bachelor of Science degree in Real-Time Interactive Simulation with a minor in Math. He has worked with numerous platforms and technologies to develop games from console to PC, but especially focusing on mobile. He has also worked on *Wolf Toss* and *Slots Tycoon* for Zipline Games using Moai. For the Moai platform, he developed various features such as the Native Client host as well as optimized and debugged it for mobile.

I would like to thank everyone at Zipline Games for their work on the Moai SDK as well as the Moai community. Without them none of this would have been possible.

Alejandro Jimenez Vilarroya is a very experienced Spanish developer who has worked with many coding languages and tools during his career.

He has been coding for fun and learning purposes since he was 7 years old, and also worked as a professional coder later on. He has worked with languages such as Lua, JavaScript, C#, and Visual Basic, developing apps and games for various desktop and mobile platforms.

Alejandro started his own company, DunkelGames, in 2010, which was co-founded with the graphic artist Joan Carles Quintans after releasing *ChocoRun*, their first and very acclaimed mobile game.

www.allitebooks.com

www.PacktPub.com

Support files, eBooks, discount offers and more

You might want to visit `www.PacktPub.com` for support files and downloads related to your book.

Did you know that Packt offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at `www.PacktPub.com` and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at `service@packtpub.com` for more details.

At `www.PacktPub.com`, you can also read a collection of free technical articles, sign up for a range of free newsletters and receive exclusive discounts and offers on Packt books and eBooks.

`http://PacktLib.PacktPub.com`

Do you need instant solutions to your IT questions? PacktLib is Packt's online digital book library. Here, you can access, read and search across Packt's entire library of books.

Why Subscribe?

- Fully searchable across every book published by Packt
- Copy and paste, print and bookmark content
- On demand and accessible via web browser

Free Access for Packt account holders

If you have an account with Packt at `www.PacktPub.com`, you can use this to access PacktLib today and view nine entirely free books. Simply use your login credentials for immediate access.

www.allitebooks.com

www.allitebooks.com

Table of Contents

Preface **1**

Chapter 1: Why Moai SDK? **7**

Multi-platform development **7**
Why Moai SDK? **8**
Summary **9**

Chapter 2: Installing Moai SDK **11**

Getting the latest stable build **11**
Setting up the environment **12**
Windows 13
Mac OS X 13
GNU/Linux 14
Running samples **14**
Summary **16**

Chapter 3: Basic Moai SDK Concepts **17**

The Moai SDK Runtime **17**
Input Queue 18
Action Tree 18
Node Graph 18
Rendering **19**
Moai Hosts **20**
Lua and C++ **20**
Summary **21**

Chapter 4: Our First Game with Moai **23**

Concentration **23**
Project setup **25**
Opening a window **25**
Resolution independence 26
Summary **28**

www.allitebooks.com

Chapter 5: Showing Images on the Screen **29**
Introduction to Decks and Props **29**
Our first image **30**
Coordinate systems **32**
Window/Screen coordinates 33
World coordinates 33
Model coordinates 33
Blend modes **34**
Summary **35**
Chapter 6: Resource Manager **37**
Resource definitions **37**
Resource manager **39**
Getter 40
Loader 40
Images 42
Tiled images 44
Fonts 45
Sounds 46
Exercises **47**
Summary **47**
Chapter 7: Concentration Gameplay **49**
Grid **49**
Tilemaps 49
The implementation 50
Input **54**
Gameplay **55**
Initialization 55
Input processing 56
Choosing a cell 58
Tile swapping 60
Resetting tiles 60
Removing tiles 61
Other auxiliary methods 61
Summary **62**
Chapter 8: Let's Build a Platformer! **63**
Background **63**
Camera and scrolling **65**
Parallax 66
Setting up the layers 66
Adding images to the different distances 67

Main character **69**
Animations 71
Summary **76**
Chapter 9: Real-world Physics with Box2D **77**
Creating the world **77**
Box2D body types **78**
Character versus the world **80**
Movement **81**
Keyboard input **84**
Creating the scene **85**
Exercises **87**
Summary **87**
Chapter 10: Creating a HUD **89**
The basics **89**
Left or right, that's the question **90**
Updating information **92**
Summary **93**
Chapter 11: Let the Right Music In! **95**
Audio manager **95**
Background music **97**
Sound effects **98**
Summary **98**
Chapter 12: iOS Deployment **99**
Xcode project **99**
The host 99
Running an example **100**
Setting up our own project **101**
Pointing to the correct source code 102
Fixing source paths 104
Fixing include paths 106
Fixing linked libraries 108
Going multi-platform **109**
Running on the device **111**
Summary **112**
Chapter 13: Deployment to Other Platforms **113**
Windows **113**
Mac OS X **113**

Android 114
Google Chrome (native client) 114
Linux 114
Summary 115
Index 117

Preface

In the last few years, multiplatform development has become a must in the game developer's skill set. Moai SDK is a great free and open source solution for this. This book will introduce you to the ways of developing games with Moai SDK using Lua; after reading it, you should be good to go and create the next hit mobile game!

We hope to see you around soon.

Welcome to Moai SDK!

What this book covers

Chapter 1, *Why Moai SDK?*, discusses the importance of multi-platform development and the advantages of using Moai SDK.

Chapter 2, *Installing Moai SDK*, helps us install Moai SDK on Windows and Mac OS X in a step-by-step manner.

Chapter 3, *Basic Moai SDK Concepts*, discusses the Action Tree, Node Graph, and Input Queue.

Chapter 4, *Our First Game with Moai*, starts work on a *Concentration*-like game. It discusses the gameplay and implements the entry point for our game.

Chapter 5, *Showing Images on the Screen*, introduces the notions of Decks and Props and guides you in showing your first image on the screen.

Chapter 6, *Resource Manager*, guides you through the creation of a Lua module that will be in charge of interfacing between your game and your assets. It will be used to cache textures, sounds, and fonts, and to avoid loading the code repeatedly.

Chapter 7, *Concentration Gameplay*, teaches us how to use grids and tilesets, and how to process input.

Chapter 8, *Let's Build a Platformer!*, guides you through the first steps in making a platformer prototype. You'll learn how to implement a layered background with parallax effects and a main character with several animations on it.

Chapter 9, *Real-world Physics with Box2D* teaches us how to use Box2D as our physics engine, and implement a basic physics world.

Chapter 10, *Creating a HUD*, teaches us how to handle fonts and display them on screen. We'll use them to show some debug info, but you can go anywhere from here.

Chapter 11, *Let the Right Music In!*, guides you in how to load audio files and play them.

Chapter 12, *iOS Deployment*, presents a case study on how to deploy to mobile devices, and guides you through all of the steps needed to make our *Concentration* game work on iOS.

Chapter 13, *Deployment to Other Platforms*, discusses the different platforms that are officially supported by Moai and points you in the right direction on how to deploy to them.

What you need for this book

For this book you'll need to have either Windows XP (or newer) or Mac OS X.

You'll need to download and install Moai SDK and ZeroBrane Studio (refer to *Chapter 2*, *Installing Moai SDK*).

Who this book is for

Moai SDK is a minimalist framework for professional game developers. This means that some of its features may be hard for complete beginners. However, this book aims to help beginners get into Moai SDK. You'll need some basic understanding on programming (Lua knowledge is not *required* but extremely recommended) and basic command-line familiarity (in order to run our games).

Conventions

In this book, you will find a number of styles of text that distinguish between different kinds of information. Here are some examples of these styles, and an explanation of their meaning.

Code words in text are shown as follows: "We're creating a method called `Game:start()` that will take care of initializing everything and controlling the game loop."

A block of code is set as follows:

```
module("Game", package.seeall)
GRID_COLS = 5
GRID_ROWS = 4
GRID_TILE_WIDTH = 62
GRID_TILE_HEIGHT = 62
BACK_TILE = 1
```

When we wish to draw your attention to a particular part of a code block, the relevant lines or items are set in bold:

```
WORLD_RESOLUTION_X = 320
WORLD_RESOLUTION_Y = 480
SCREEN_RESOLUTION_X = 2 * WORLD_RESOLUTION_X
SCREEN_RESOLUTION_Y = 2 * WORLD_RESOLUTION_Y
```

New terms and **important words** are shown in bold. Words that you see on the screen, in menus or dialog boxes for example, appear in the text like this: "Open up the ZeroBrane Studio and on the left panel (**Project**), click on the button with the ellipsis (**...**) and head to `moai-sdk/samples/anim/anim-basic`, and click on **OK**".

Warnings or important notes appear in a box like this.

Tips and tricks appear like this.

Reader feedback

Feedback from our readers is always welcome. Let us know what you think about this book—what you liked or may have disliked. Reader feedback is important for us to develop titles that you really get the most out of.

To send us general feedback, simply send an e-mail to `feedback@packtpub.com`, and mention the book title via the subject of your message.

If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, see our author guide on `www.packtpub.com/authors`.

Customer support

Now that you are the proud owner of a Packt book, we have a number of things to help you to get the most from your purchase.

Downloading the example code

You can download the example code files for all Packt books you have purchased from your account at `http://www.packtpub.com`. If you purchased this book elsewhere, you can visit `http://www.packtpub.com/support` and register to have the files e-mailed directly to you.

Errata

Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you find a mistake in one of our books—maybe a mistake in the text or the code—we would be grateful if you would report this to us. By doing so, you can save other readers from frustration and help us improve subsequent versions of this book. If you find any errata, please report them by visiting `http://www.packtpub.com/submit-errata`, selecting your book, clicking on the **errata submission form** link, and entering the details of your errata. Once your errata are verified, your submission will be accepted and the errata will be uploaded on our website, or added to any list of existing errata, under the Errata section of that title. Any existing errata can be viewed by selecting your title from `http://www.packtpub.com/support`.

Piracy

Piracy of copyright material on the Internet is an ongoing problem across all media. At Packt, we take the protection of our copyright and licenses very seriously. If you come across any illegal copies of our works, in any form, on the Internet, please provide us with the location address or website name immediately so that we can pursue a remedy.

Please contact us at `copyright@packtpub.com` with a link to the suspected pirated material.

We appreciate your help in protecting our authors, and our ability to bring you valuable content.

Questions

You can contact us at `questions@packtpub.com` if you are having a problem with any aspect of the book, and we will do our best to address it.

1
Why Moai SDK?

Over the last two decades we've witnessed explosive development in software platforms, mostly represented by the mass adoption of Linux and Mac OS, and in the last few years, Android and iOS. In this chapter we'll see what this entails and why Moai SDK is here to help us all, and that for free.

Multi-platform development

So you want to make games. You probably have this great game idea and you're planning to release it for the Web, iOS, and Android platforms, and probably if you get traction, publish it on Steam. I bet you already have some concept art, or your **Game Design Document** (**GDD**) in place. But, have you planned on how are you going to make your game in order to be able to support all those platforms?

Nowadays you have three options to handle this problem:

- Build the game *n* times, where *n* is equal to the number of platforms
- Forget about multi-platforms and focus on one or two
- Use a library/framework designed to support multiple platforms

The first option has an advantage in that you can optimize your game for each platform, ensuring it runs smoothly. Also, you can take care of platform limitations and change how your game looks according to them. But have you thought about the implications of having, say, four different code bases? Each one with their own bugs and ways of implementing logic. Additionally, you may need to hire multiple programmers if you can't find one with in-depth knowledge of all platforms. It's a nightmare.

www.allitebooks.com

Forgetting about multi-platforms could be a good choice for you, but in the end, if you make a really successful game (and a probable way of making a game successful is to have it available on as many platforms as possible, since this increases awareness), you'll want to port it to different platforms at low cost, and porting is expensive if the code is extremely tied to a specific platform.

This leads us to the last option, embrace a library or framework that supports multi-platform development. The basic idea is that you detach your code from specific platforms, using a widely-spread language (such as C, C++, JavaScript, or Lua) and build your game for different platforms using a single code base.

A real-life example on the benefits of multi-platform development is Zipline Games' *Wolf Toss*. It had a 3-month development cycle and was released simultaneously on iOS and Android. If it were to be built for each platform, it'd have taken about three months for each platform, or a team twice as big. So, by developing *Wolf Toss* with Moai SDK, the studio spent half of the money that it would take with a platform-specific approach.

This approach has been gaining popularity lately because of its simplicity and powerful consequences. Many game development environments and frameworks that take this approach have seen the light of day in the last few years. One of them is Moai SDK.

Why Moai SDK?

It's safe enough to say that almost every single platform nowadays supports C++ and some flavor of OpenGL. That's the starting point for Moai SDK.

Moai SDK is a multi-platform game development framework. It's been developed in C++ and all of its features are accessible through Lua. What does this mean? That you can build your game completely in Lua (taking advantage of the flexibility of this amazing language) and, in case you need low-level access, you can always switch to C++ and do whatever you want. You can even bind your C++ libraries to use them in your game, in Lua. It has built-in access to Moai Cloud as well, which is a cloud service that allows you to deploy server-side code written in Lua, with databases, support for push notifications, leaderboards, and other fancy stuff. Note that the language is the same for your game and for your server-side code, nice.

In short, *if XYZ platform supports OpenGL and C++, Moai SDK is likely to be supported by it*.

Currently Moai SDK works on Windows, Mac, Linux, iOS, Android, and Native Client.

Anyway, there are many game engines that support multi-platform development. Many of them come with flashy interfaces, others with half of your problems solved, such as one-click deployment. So, what does Moai SDK bring to this world that has not been done yet? *Freedom*.

- It's open source, you can download the code and do whatever you want with it.
- It's free, no hundreds of dollars in licenses.
- It doesn't tell you how to do things. It's aimed at pro developers who want to do things their way.
- It has been used by several large-scale commercial games.

So these are the main principles behind Moai SDK's philosophy. A great mix for a great game.

Summary

Nowadays, to be able to develop games it's important to take into account multi-platform development. Moai SDK is a game development framework that helps us deal with this problem in an easy way using standard technologies that are available in almost every platform. Not only that, it's *free* and *open source*.

2
Installing Moai SDK

Before we can get our hands dirty, there are a couple of steps we need to take in order to have a fully working environment to develop in with Moai SDK.

This differs from platform to platform, so feel free to jump to your platform of choice.

We'll see how to install Moai SDK in Windows and Mac OS X; for Linux users, the process is more complicated and we can't cover it here, but you'll find more information about it in this chapter as well. At the end of the chapter, you should end up with a working copy of Moai SDK installed on your system.

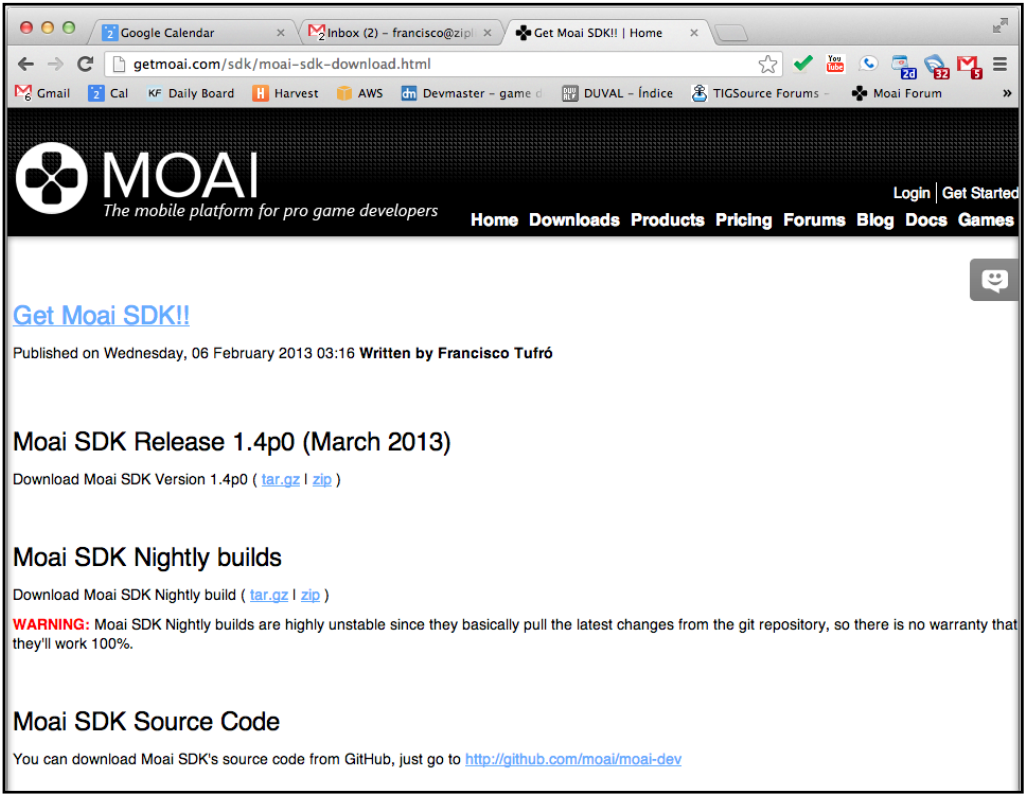
Getting the latest stable build

Head to `http://getmoai.com/sdk/moai-sdk-download.html`.

You'll find two download options, the release and the nightly build.

- **Release**: This is a stable version of Moai SDK. It includes all you need to start developing with Moai SDK. The current version at the time of this writing is 1.4p0.
- **Nightly builds**: Every night (or morning depending on where are you located!) there is an automatic build of the whole Moai SDK project. This build is extremely unstable since it includes all the changes since the last stable release. If you need some feature that has not been included in the latest release, download the nightly build, it should be there.

For each of these versions, there are two possible packages: a `.zip` and a `.tar.gz` package, download the one you like the most.



Setting up the environment

After downloading the SDK, we need to set up our environment.

Moai SDK can be used with any text editor, but if you want some fancy features such as a debugger, you'd need to use one of the IDE's that support Moai SDK. There are a bunch of them, each with different levels of integration. For a complete list, I suggest you visit *Tools that work with Moai* on Moai's wiki at `http://getmoai.com/wiki/index.php?title=Tools_that_work_with_Moai`.

In this book we'll focus on using ZeroBrane Studio, since it's open source (you can program it in Lua) and has a lot of neat features that will be useful to us (syntax highlighting, autocompletion for Moai SDK, debugging, and live coding, among others). To install it, go to `http://studio.zerobrane.com` and download the latest version of the software for your specific platform.

Windows

The book assumes that you've already downloaded ZeroBrane Studio (either the zip or the installer) and installed it in `C:\Program Files\ZeroBraneStudio`.

Now, unzip the Moai SDK package to `C:\moai-sdk`.

After doing this, we need to set up ZeroBrane to work with our Moai SDK installation.

1. Create a file called `user.lua` in `C:\Program Files\ZeroBraneStudio\cfg\` and add the following lines:

   ```
   -- MOAI Path
   path.moai = 'c:/moai-sdk/bin/win32/moai.exe'
   ```

2. If you chose to install Moai SDK in another location use that location instead.
3. There is one last thing you need to do in order to support Moai SDK.
4. Navigate to **Project** | **Lua Interpreter** | **Moai**.

Now you're all set. You can jump to the *Running Samples* section to see if everything worked correctly.

Mac OS X

The book assumes that you've already downloaded ZeroBrane Studio for Mac OS X and have installed it in `/Applications`.

Now, unzip the Moai SDK package to `~/moai-sdk`.

1. Create a file called `user.lua` in `/Applications/ZeroBraneStudio.app/Contents/ZeroBraneStudio/cfg/` and add the following lines:

   ```
   -- MOAI Path
   path.moai = '~/moai-sdk/bin/osx/moai'
   ```

2. If you chose to install MOAI in another location use that location instead.
3. The last step is to navigate to **Project** | **Lua Interpreter** | **Moai**.

Now you're all set.

The next step is to try some samples to see if everything is working correctly.

GNU/Linux

Sadly, the current stable release of Moai SDK doesn't include a precompiled host for GNU/Linux. So the only option is to build your own host from the source code. In order to do this, you'll need some understanding of how to compile C++ and how to handle the Git repositories (at least how to clone them).

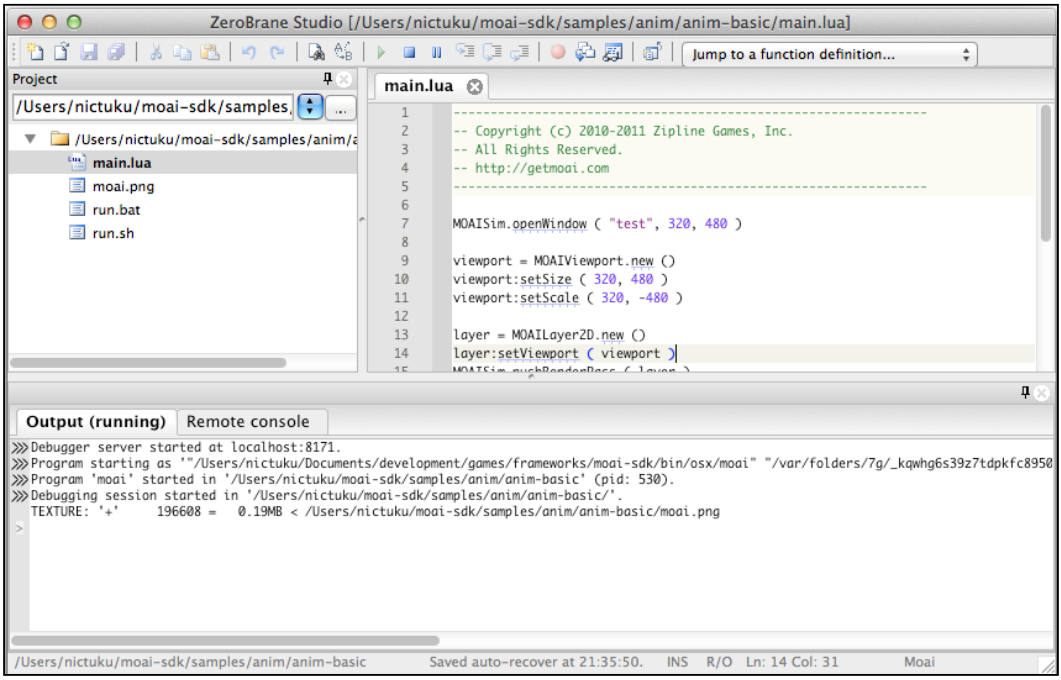
This is outside the scope of this book, but feel free to jump into the Moai forums to get help if you're lost.

Just as a hint, you'll need to get the source code from `https://github.com/moai/moai-dev`, switch to the Linux branch and compile everything using *CMake*.

Running samples

In order to see if your installation works fine, we will run a sample from Moai's release.

1. Open up the ZeroBrane Studio and on the left panel (**Project**), click the button with the ellipsis (**...**) and head to `moai-sdk/samples/anim/anim-basic`, and click **OK**.



Now your project is using one of Moai SDK's samples as project root. You'll see a lot of examples on how to do stuff with Moai SDK in the `moai-sdk/samples/folder`. Take a look at them after reading the book in order to understand more advanced topics.

2. Now press *F6* (or **Project | Run**).

   If a window opens up and shows Moai SDK's logo rotating, yay! You're ready to go!

You will want to read the sample; especially take a look at the line that reads:

```
prop:moveRot ( 360, 1.5 )
```

This line is what makes the Moai logo move. The first parameter is the rotation angle and the second is the time it takes to perform the whole movement. Go ahead, play with it, change the angle and duration time and see what happens.

Summary

In this chapter, we've seen that in order to develop with Moai SDK, we need to use one of the precompiled hosts or build one from source (in the case of the Linux development this is mandatory, since there are no precompiled builds yet).

We've downloaded the latest version of the framework, installed it, and run a sample to see if it's working properly.

The next chapter will introduce you to the basic concepts behind Moai SDK, so keep reading if you want to learn about Moai's runtime, the Action Tree, and Node Graph.

3
Basic Moai SDK Concepts

Every game engine has its own concepts behind it that define its internal workings. In this chapter, we will take a quick tour of the different ideas that Moai SDK uses, in order to understand it better.

The Moai SDK Runtime

Moai's Runtime uses two important data structures, namely, a *dependency graph of objects/nodes* and a *tree of actions*. What you'll (mostly) do as a developer is modify those data structures (in Lua, using the Moai SDK API).

The Moai update loop has three steps:

- Process the Input Queue
- Process the Action Tree
- Process the Node Graph

In Moai, *rendering is detached from the game loop*, since we might want to have more than one update step for every rendered frame.

www.allitebooks.com

Input Queue

When you press a key, move the mouse, or tap on your iPhone's screen, the system generates an input event that is passed to Moai by the **Host** and put into a queue (Using AKUEnqueue* methods). In every step of the simulation, Moai handles all input callbacks, processes the Input Queue, and then continues with the game loop.

All the events that occur during the game loop are enqueued in the next simulation step.

Action Tree

Nodes are basically all of the objects that have information for your game, for example, an instance of a bullet would be a node in Moai, specifically a node that can be rendered.

Actions are in charge of managing the node's variation over time.

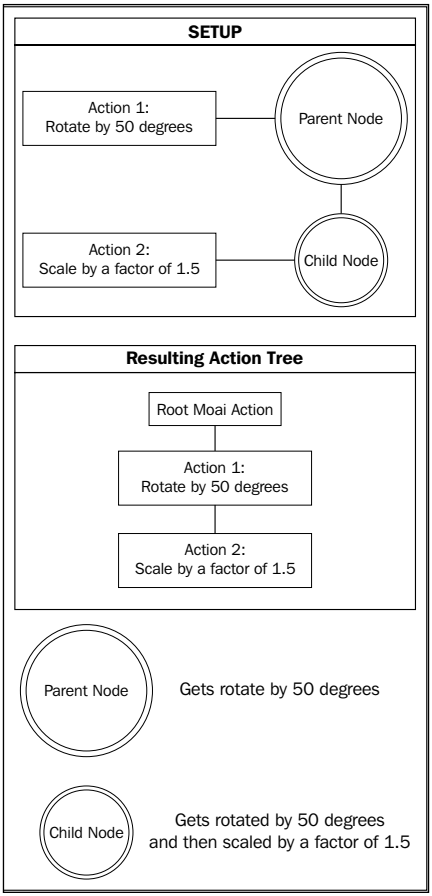
For example, changes in position, rotation, or scale would create an animation.

The role of the the Action Tree is to create a hierarchy of actions. The main goal for this is to enable actions to be responsible for the execution of child actions. Moai has a main root action that will run all of the actions appended to it.

Actions are the only objects in Moai that receive time steps. This means that time-based changes to Moai objects may only occur while processing the Action Tree. The Node Graph or rendering processes should not perform any change to the state of nodes.

Node Graph

So, as we saw before, nodes are basically the objects that are living in your game. These objects connect with one another forming a graph. In every update loop step, Moai processes the nodes that have different states than the previous step. This is achieved with a queue of modified nodes. Every time an action modifies a node (or the node is linked to another node that was modified, or when a node's property was set directly), it's scheduled (along with the nodes it's connected to, and so on) and processed in the next simulation step.



This behavior defines a parent/child relationship that is built at a low level, and thus allows easily created scene and animation hierarchies.

Rendering

So, where does the rendering take place then? In Moai SDK, rendering is detached from the update loop. The Render Manager is a singleton that is in charge of rendering. It keeps a table of objects to be rendered, you can modify that table as you want, and it will render all of the objects in it. It is worth mentioning that subtables are also rendered, so you can create a hierarchy of objects to be rendered.

Moai Hosts

One of the key concepts behind Moai SDK is that of **hosts**.

This is a neat solution that the folks from Zipline proposed for Moai SDK and basically states that Moai SDK provides a Lua interpreter and a C++ interface to the outside world. This interface is called **AKU**, and can be found in the Moai SDK source code.

Whenever you want to create a game for a specific platform, your host is in charge of configuring the input and sending input events to Moai, solving any threading issues from that platform, and also providing all platform-specific logic and Lua extensions along with an OpenGL canvas for rendering.

In this way, a game built in Moai is potentially deployable to all platforms that support C++ and OpenGL (well, not all OpenGL, just a subset of it, such as OpenGL ES), and those being industry standards, there is hardly any limitation.

Moai SDK is shipped with sample hosts for Windows, Mac OS X, iOS, Android, and Google's Native Client. You should be able to modify these hosts, and with little tweaks, have your game working on them.

Again, you build your game once using Lua, and you'll get all of these platforms to run on.

Lua and C++

Another interesting feature behind Moai is the that you have access to both Lua and C++ to develop your game.

After setting up your nodes and actions using Lua, Moai SDK runs the simulation using native code written in C++. This means that you get the best of both worlds—the flexibility of Lua and the speed of C++.

All the platform-specific stuff (for example, handling the accelerometer on a smartphone) should be handled natively and connected to Moai SDK using a Lua extension or input events (check how hosts configure AKU, search for the AKUSetInputDevice* family of methods).

This is a key concept for Moai SDK development, because you can see that there is no limit to what libraries you can use with Moai SDK. It's just a matter of taking the time to write a Lua extension for it, and Bam! It's available in the scripting environment; you can combine the library functionality with your already existing game.

Aside from this, there is always the interpreted versus compiled native code dilemma. You know that working with Lua is more productive than using C++ for all the coding, but there are times when you just can't do something in Lua because of performance requirements. (Although this is extremely rare, Lua is fairly fast and bottlenecks usually are to be found in rendering, logic execution takes very little time). You can always port your code to C++ (you can decide whether this is needed using profiling tools), then create a Lua extension with the same API, and you won't need to modify the rest of your code.

Summary

In this chapter, we talked about the nuts and bolts of the Moai SDK update loop in order to understand its inner workings. We took a look at the three main steps in the update loop, namely, the Input Queue, processing of the Action Tree, and processing of the Node Graph. We talked about Moai's detached rendering nature, the importance of hosts, and the idea behind having the C++ and Lua worlds tied together, and how to make wise use of that.

Well, enough talk about what's going on behind the scenes. Get a coffee now, since in the next chapter, we'll finally start implementing our first game and your curiosity won't let you take a break then.

4

Our First Game with Moai

Adventure time! We're going to start working on our first game made with Moai SDK, a simple *Concentration*-like game. For this you'll need some understanding of Lua syntax, nothing too fancy, just the basics. If you've never used Lua before, it'd be a good idea to read some quick introduction material and have a syntax cheat sheet around when reading this book.

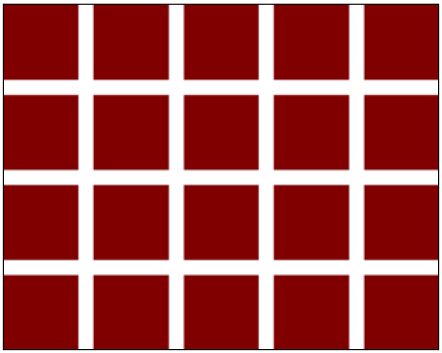
Concentration

For those readers who are not familiar with this game, let's describe it.

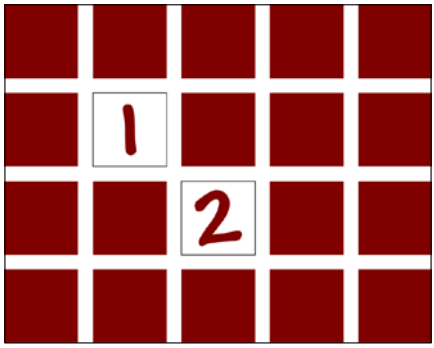
The game requires a set of tiles. There must be an even number of them. The tiles have figures (in pairs, since we're going to match them) on one side and a flat color or identical image on the other side (the back of the tile).

The gameplay is as follows:

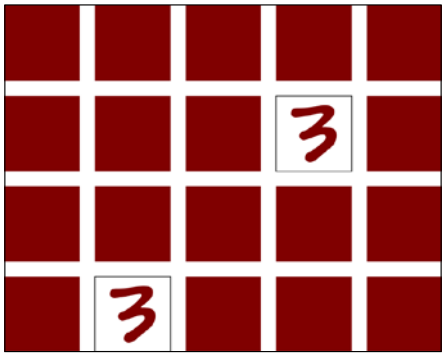
1. Place all the tiles facing down and shuffle them. This way you hide the pairs and have no idea where they are.



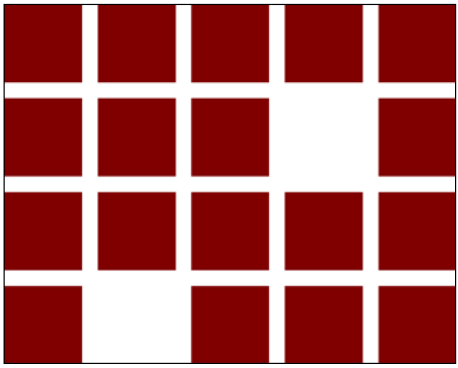
2. Then, you have to choose two tiles and flip them, to reveal the images.



3. If they're different, you have to flip them back again and pick a different pair.



4. If they are the same, you remove them from the board and the game continues with two tiles less.



The game ends when there are no tiles left.

We will implement this game, with the addition of a ticking clock, so things get a little bit more challenging.

In order to do this we'll examine all of Moai SDK's basic features and how to use them.

Project setup

To set up the project we need to perform a couple of steps:

1. Create a folder for our project somewhere on your hard drive.
2. After that open ZeroBrane Studio and navigate to that folder.
3. You'll now see an empty folder and a blank file named `untitled.lua`.
4. Save that file as `main.lua`, which is the usual name for a game entry point file. You can name it whatever you want, but we'll use this as a standard.
5. Now we are ready to start.

Opening a window

The first thing we'll learn is how to open the main game window using Moai SDK.

This is a relatively easy task. The code is as follows:

```
MOAISim.openWindow ( "Concentration", 320, 480 )
```

What we are doing here is calling the `MOAISim.openWindow` class method with the necessary parameters, namely, `title` for window (a string), `width`, and `height`. As you may have noticed, we used 320 x 480, which is the display resolution of an iPhone 3GS (without Retina Display).

If you run the script now, you'll see that a window opens. So that's it, our main window is ready. But before going further, let's take a look at an important aspect of Moai SDK.

Downloading the example code

You can download the example code files for all Packt books you have purchased from your account at `http://www.packtpub.com`. If you purchased this book elsewhere, you can visit `http://www.packtpub.com/support` and register to have the files e-mailed directly to you.

Resolution independence

The window we just opened has a specific width and height. But, as we know, different devices have different resolutions and we can't just open up a screen with a fixed width and height because it won't behave correctly on say, an iPhone and an iPad.

So we'll introduce a concept here **world resolution** versus **screen/window resolution**.

The idea is that your game will use world resolution for all the calculations and object placement, and then you'll use a viewport that understands that world resolution and translates it into window resolution. So, let's set that up our script as follows:

```
WORLD_RESOLUTION_X = 320
WORLD_RESOLUTION_Y = 480

SCREEN_RESOLUTION_X = 2 * WORLD_RESOLUTION_X
SCREEN_RESOLUTION_Y = 2 * WORLD_RESOLUTION_Y

-- Open main screen
MOAISim.openWindow ( "Concentration", SCREEN_RESOLUTION_X,
SCREEN_RESOLUTION_Y )

-- Setup viewport
viewport = MOAIViewport.new ()
viewport:setSize ( SCREEN_RESOLUTION_X, SCREEN_RESOLUTION_Y )
viewport:setScale ( WORLD_RESOLUTION_X, WORLD_RESOLUTION_Y )
```

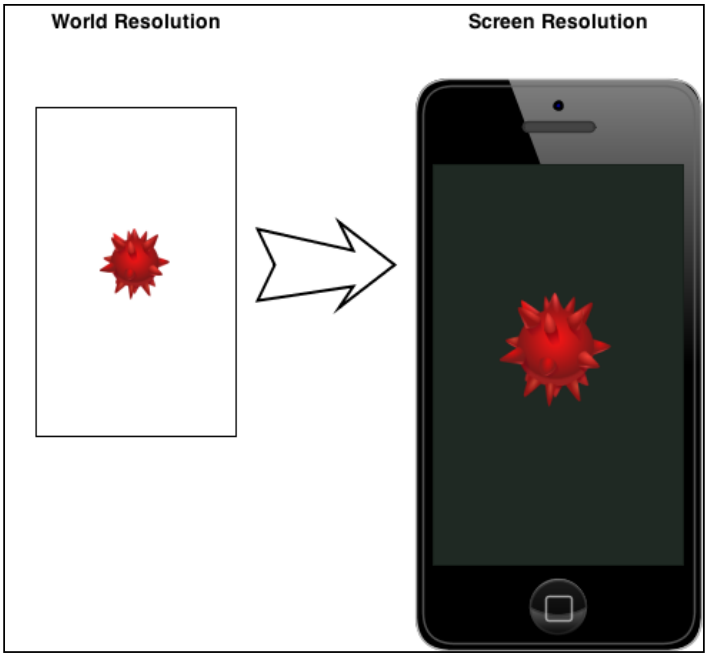
Let's take a look at the preceding code line by line:

```
WORLD_RESOLUTION_X = 320
WORLD_RESOLUTION_Y = 480
```

These lines are defining two constants, namely, our world resolution along the X and Y axes. Remember that these are not the dimensions of the screen, just our world.

```
SCREEN_RESOLUTION_X = 2 * WORLD_RESOLUTION_X
SCREEN_RESOLUTION_Y = 2 * WORLD_RESOLUTION_Y
```

Now we define `X` and `Y` dimensions for our screen's resolution. As you can see, we made our screen double the size of our world. We do this is in order to better illustrate the independence of screen resolution with respect to world resolution. In the following chapters, we'll see that anything you place in the world will look twice as big on the screen as follows:



We now need to tell Moai to open a window, this is needed during development to create a window for our game, on the devices it will just do nothing.

```
MOAISim.openWindow ( "Concentration", SCREEN_RESOLUTION_X,
SCREEN_RESOLUTION_Y )
```

This is the same line we saw before, but now, we are using our `SCREEN_RESOLUTION_X` and `SCREEN_RESOLUTION_Y` constants. Note that in devices that have fixed size screens, this method won't do anything special, but we need it to run our samples in the computer.

```
viewport = MOAIViewport.new ()
```

Now, here is something new (literally). In order to handle the conversion between world and screen coordinates, we will use the `MOAIViewport` instances. Note that you may have an arbitrary number of viewports, each using different coordinate scales. For example, we could use another viewport with the same scale as the window for some specific effect or for a user interface (GUI) that doesn't need scaling.

```
viewport:setSize ( SCREEN_RESOLUTION_X, SCREEN_RESOLUTION_Y )
```


www.allitebooks.com

Now we set the size for the viewport. This is usually the same size as the screen resolution (additionally, on devices this must match the screen resolution if you want it to take up the entire screen), but you could make a smaller viewport in order to display a small map or something like that, but keep in mind that the whole idea of the size is that it will be the target resolution.

```
viewport:setScale ( WORLD_RESOLUTION_X, WORLD_RESOLUTION_Y )
```

And this is what completes the magic. We set a scale, using a different resolution. If you think about this, what we're doing is simply telling the viewport that whatever we put on our layers in the future, needs to be stretched/shrunk from the world coordinates (`WORLD_RESOLUTION_X` and `WORLD_RESOLUTION_Y`) to another set of coordinates (`SCREEN_RESOLUTION_X` and `SCREEN_RESOLUTION_Y`).

That's it, we now have a resolution-independent screen. It's just a matter of changing `SCREEN_RESOLUTION_X` and/or `SCREEN_RESOLUTION_Y` and the game will adjust accordingly.

Summary

So, we are now committed to making a *Concentration* game using Moai SDK. We reviewed the rules and goals of the *Concentration* game. We also wrote our first lines with Moai SDK to open a window and discussed how to implement resolution-independent games using world versus screen coordinate systems.

The next chapter will teach you how to display images on screen, the most basic skill for today's game developers.

5
Showing Images on the Screen

One of the basic skills we need to obtain in order to develop games is **image manipulation**. In this chapter, we'll discuss some aspects of this topic that will be useful for our game. We'll introduce the notion of Decks and Props and talk about how to handle them in order to make our game graphics.

Introduction to Decks and Props

In Moai SDK, all of the assets (images, tilesets, meshes, and surfaces) are defined in Decks. You can think of a Deck as the geometry of an object. **Decks** hold the actual image data for a sprite, for example. They're not shown in the screen, until you reference them using a Prop. You can think of Decks as the Prop's drawing method.

There are many types of Decks `MOAIGfxQuad2D`, for example, handles one single image. `MOAITileDeck2D` is used to load a tileset (or atlas) and reference its tiles (we will use it later in the book).

Props are basically a collection of information and a reference to a given Deck. They hold information such as location, scale, and rotation; and are the actual objects that you see on the screen.

This separation is pretty useful, since it allows you to reuse the memory allocated to a specific asset. We will use this to handle all of our tiles and instead of loading the image for the back of the tile 20 times, we'll just load it once and point to it.

Our first image

The first thing we need to do is to have an actual image. We'll use the back of a tile from our *Concentration* game in this example, you can get it from the book's website, along with the full source code for this example.

To begin with, we need a layer. A **layer** groups together one or more Props and serves as a way to control rendering priorities between different Prop sets. For example, you can have one layer for the background and another one on top of it for the objects, and when you add a new Prop on the background layer, it will be displayed below the Props on the objects layer.

> `MOAILayer` inherits from `MOAIProp`, so you can think of it as a Prop that groups other Props. You can call any of the methods of `MOAIProp` on a layer.

In order to create a layer, you have to instantiate `MOAILayer2D` as follows (type this below the viewport initialization in the code for *Chapter 4*, *Our First Game with Moai*):

1. First of all we create our layer:

   ```
   layer = MOAILayer2D.new ()
   ```

2. We need to tell that layer to use our viewport. This is done using the `setViewport()` method.

   ```
   layer:setViewport ( viewport )
   ```

3. Now we have to create our Deck. Since we're only displaying a simple image, we will use `MOAIGfxQuad2D` which serves that purpose.

   1. We create the Deck object.

      ```
      imageGfx = MOAIGfxQuad2D.new ()
      ```

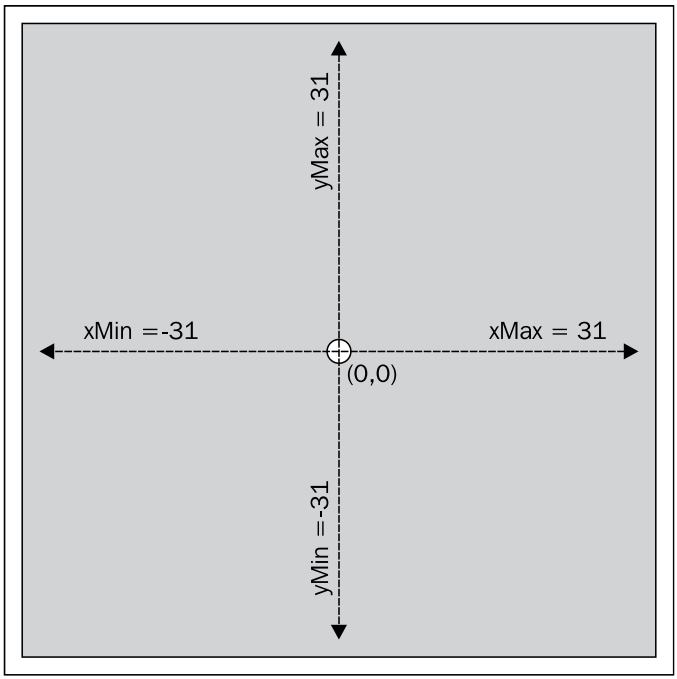
   2. We use the `setTexture()` method and pass the path of our asset to it (in this case the back of a tile). This is where we tell the Deck to actually use our image.

      ```
      imageGfx:setTexture ( 'assets/tile_back.png' )
      ```

   3. The third line sets the geometry. `MOAIGfxQuad2D` has a rectangular geometry. In order to define it, we can use `setRect`, which receives four numbers, `xMin`, `yMin`, `xMax`, and `yMax`.

      ```
      imageGfx:setRect ( -31, -31, 31, 31 )
      ```

We'll use the convention that the object coordinate `(0, 0)` is in the center. In this way, since we want to show the image with a 62 x 62 size, the actual rectangle that holds it can be defined as the rectangle that has `xMin = -31`, `yMin = -31`, `xMax = 31`, and `yMax = 31`, since we want it to have its `(0,0)` at the center of the image. We could define it with `setRect ( 0, 0, 62, 62 )` and the center would be at the upper-left corner, but let's keep `(0,0)` at the center of the image convention for simplicity. Just note that this is pretty powerful. You could define the rectangles differently according to how you want to manipulate them.



4. The next step is to set up our Prop.
    1. First of all we create Prop.

       ```
       imageProp = MOAIProp2D.new ()
       ```

    2. Then we tell the Prop to use `imageGfx` as the Deck. This is the point where we connect them together. We're saying that this prop will be an instance of our back of tile image.

       ```
       imageProp:setDeck (imageGfx)
       ```

    3. The third line moves the Prop to the center of the screen.

       ```
       imageProp:setLoc (0, 0)
       ```

We're using the `setLoc()` method to move the image to the back of the tile. You can change the parameters and see how it behaves. By default, `(0,0)` is the center of the screen, so negative values in *x* will move the object to the left and positive ones to the right. Negative values in *y* will move the object down and positive ones will move it up. This is not the same as the typical coordinate system used in computer graphics, which sets `(0,0)` at the top-left corner and the *y* axis grows downward.

5. The next step is to insert our Prop into the layer we created, so it will render when the layer is rendered.

```
layer:insertProp (imageProp)
```

6. And the last step is to make that layer renderable.

```
renderTable = { layer }
MOAIRenderMgr.setRenderTable (renderTable)
```

This is an interesting thing to talk about. What we are doing here is, first, creating a table that contains the layer as its first object, and then telling `MOAIRenderMgr` to use that table for rendering.

In order to configure rendering, we need to tell `MOAIRenderMgr` what we want to render. This is done using the **Render table**. As we discussed in *Chapter 3*, *Basic Moai SDK Concepts*, the table and subtables that you pass using `setRenderTable()` are the ones that will be rendered during the next rendering step. So, for example if you want to remove all of the objects that are rendering, you don't need to iterate through them, you just have to pass an empty table to the `setRenderTable()` method.

You have to be careful when handling this. Keep an eye on the table you're rendering if objects suddenly disappear.

Coordinate systems

Sooner or later, if you're developing multi-platform games, you'll need to understand the different coordinate systems that are involved.

In Moai SDK, we have at least three coordinate systems that are important to remember and to make good use of.

Window/Screen coordinates

This is the coordinate system attached to your window. You will typically have to deal with it when receiving mouse or touchscreen input. You might also use it to design your GUI. If you ever experience any strange behavior when clicking, always double-check that you're correctly transforming input coordinates to world or even model coordinates.

World coordinates

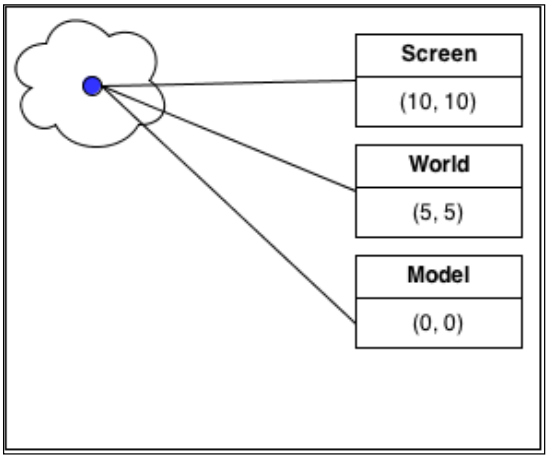
This is the coordinate system that you define when setting the scale for the viewport, and is the one that you'll normally use when positioning Props on the screen. This is where the multi-platform magic happens, since this coordinate system is the one that abstracts different screen resolutions so that you don't need to take them all into account. Aspect ratio changes need to be handled, but that's a separate topic by itself.

Model coordinates

This coordinate system is the one Props use. It's related to Prop geometry.

It's really useful when checking for collisions. Given an arbitrary point in the world, converting it to model coordinates will give you a point that has a certain offset in x and y from the Prop's center. If you know the Prop's size, you can actually check if the point is inside the Prop's bounds.

Changing between coordinate systems can simplify your game's logic a lot.



Luckily, this coordinate nightmare is really easy to address in Moai SDK.

There are some methods that will help us in this task:

- `MOAILayer2D` has methods to transform between window and world coordinates (`wndToWorld` and `worldToWnd`).
- `MOAIProp2D` has methods to transform between world and model coordinates (`modelToWorld` and `worldToModel`).

So, you will want to use these methods in order to do your calculations based on what entity you're working with. We will use this when implementing the gameplay for our *Concentration* game.

Blend modes

A nice feature that you'll be playing with for sure is **blend modes**.

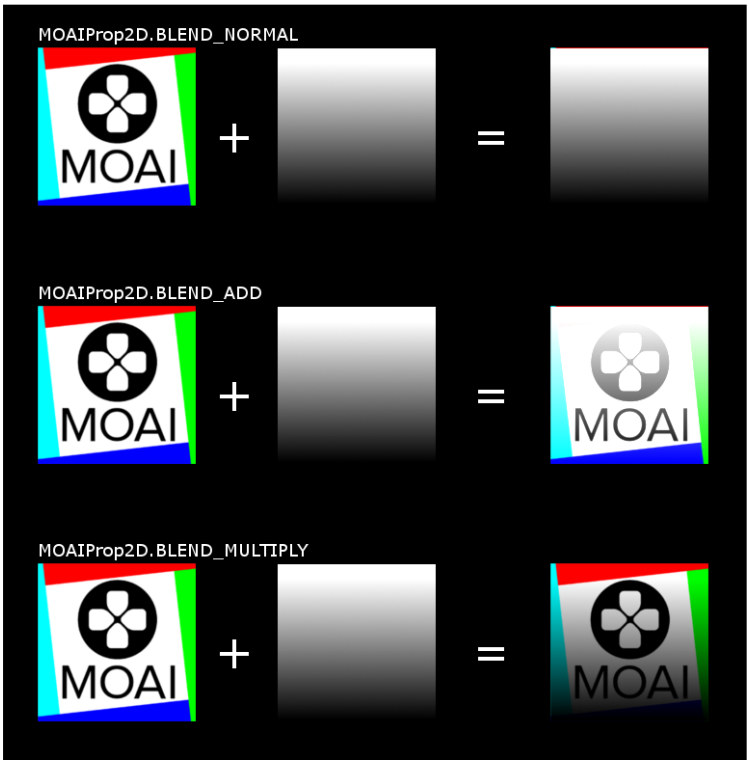
Moai SDK gives you the ability to define the blending mode for a certain Prop using the `setBlendMode()` method. This will allow you to modify the way its pixels are rendered on the screen.

You can choose between `MOAIProp2D.BLEND_NORMAL`, `MOAIProp2D.BLEND_ADD`, and `MOAIProp2D.BLEND_MULTIPLY`.

`MOAIProp2D.BLEND_NORMAL` is the default behavior and basically paints over pixels, covering whatever is underneath.

`MOAIProp2D.BLEND_ADD` does additive blending. It adds the value for the pixel you're trying to draw to the previous value of the pixel on the screen. This is really useful for particle systems and to obtain that popular glowing effect used to create *Tron*-like lights. It can also be used to create static lighting for a scene.

`MOAIProp2D.BLEND_MULTIPLY` multiplies the values of the pixel on the screen and the one it's trying to paint. This is really useful for creating static shadows.



Summary

In this chapter, we learned about Decks and Props—objects that Moai SDK provides to handle images. We saw how to combine these objects, along with the viewport, and show an image on the screen. We also talked about coordinate systems and the importance of understanding them, and when to use which in order to make calculations easier (and correct!). We also introduced the usage of blend modes, which will come in handy for creating nice effects with lights and shadows.

The next chapter will be the basis of our minimalist game framework, the Resource Manager. This contains code that will handle loading and caching objects in memory. We'll use it to avoid repeating initialization code when we need to use images or sounds.

6
Resource Manager

If we start developing our game, we'll eventually need to create an entity that handles all of our assets. We'll call it `ResourceManager`. We'll see how to create one that allows you to add images, fonts, and sounds to your games.

The main idea behind the resource manager is to cache assets that we'll use more than once and to have a centralized and abstracted way to create assets.

Resource definitions

In order to be able to define resources, we need to create a module that will be in charge of handling this. The main idea is that before calling a certain asset through `ResourceManager`, it has to be defined in `ResourceDefinitions`. In this way, `ResourceManager` will always have access to some metadata it needs to create the asset (filenames, sizes, volumes, and so on).

In order to identify the asset types (sounds, images, tiles, and fonts), we will define some constants (note that the number values of these constants are arbitrary; you could use whatever you want here). Let's call them `RESOURCE_TYPE_[type]` (feel free to use another convention if you want to). To make things easier, just follow this convention for now since it's the one we'll use in the rest of the book. You should enter them in `main.lua` as follows:

```
RESOURCE_TYPE_IMAGE = 0
RESOURCE_TYPE_TILED_IMAGE = 1
RESOURCE_TYPE_FONT = 2
RESOURCE_TYPE_SOUND = 3
```

If you want to understand the actual reason behind these resource type constants, take a look at the `load` function of our `ResourceManager` entity in the next section.

www.allitebooks.com

We need to create a file named `resource_definitions.lua` and add some simple methods that will handle it.

Add the following line to it:

```
module ( "ResourceDefinitions", package.seeall )
```

The preceding line indicates that all of the code in the file should be treated as a `module` function, being accessed through `ResourceDefinitions` in the code. This is one of many Lua patterns used to create modules.

If you're not used to the Lua's `module` function, you can read about it in the modules tutorial at `http://lua-users.org/wiki/ModulesTutorial`.

Next, we will create a table that contains these definitions:

```
local definitions = {}
```

This will be used internally and is not accessible through the module API, so we create it using the keyword `local`.

Now, we need to create the `setter`, `getter`, and `unload` methods for the definitions.

The `setter` method (called `set`) stores the `definition` parameter (a table) in the `definitions` table, using the `name` parameter (a string) as the key, as follows:

```
function ResourceDefinitions:set(name, definition)
    definitions[name] = definition
end
```

The `getter` method (called `get`, duh!) retrieves the definition that was previously stored (by use of `ResourceDefinitions:set ()`) using the `name` parameter as the key of the `definitions` table, as follows:

```
function ResourceDefinitions:get(name)
    return definitions[name]
end
```

The `final` method that we're creating is `remove`. We use it to clear the memory space used by the definition. In order to achieve this we assign `nil` to an entry in the `definitions` table indexed by the `name` parameter as follows:

```
function ResourceDefinitions:remove (name)
  definitions[name] = nil
end
```

In this way, we remove the reference to the object, allowing the memory to be released by the garbage collector. This may seem useless here, but it's a good example of how you should manage your objects to be removed from memory by the garbage collector. And besides this, we don't know information comes in a resource definition; it may be huge, we just don't know.

This is all we need for the resource definitions. We're making use of the dynamism that Lua provides. See how easy it was to create a repository for definitions that is abstracted from the content of each definition. We'll define different fields for each asset type, and we don't need to define them beforehand as we probably would have needed to do in C++.

Resource manager

We will now create our resource manager. This module will be in charge of creating and storing our decks and assets in general. We'll retrieve the assets with one single command, and they'll come from the cache or get created using the definition.

We need to create a file named `resource_manager.lua` and add the following line to it:

```
module ( "ResourceManager", package.seeall )
```

This is the same as in the resource definitions; we're creating a module that will be accessed using `ResourceManager`.

```
ASSETS_PATH = 'assets/'
```

We now create the `ASSETS_PATH` constant. This is the path where we will store our assets. You could have many paths for different kinds of assets, but in order to keep things simple, we'll keep all of them in one single directory in this example. Using this constant will allow us to use just the filename instead of having to write the whole path when creating the actual resource definitions, saving us some phalanx injuries!

```
local cache = {}
```

Again, we're creating a `cache` table as a local variable. This will be the variable that will store our initialized assets.

Now we should take care of implementing the important functionality. In order to make this more readable, I'll be using methods that we define in the following pages. So, I recommend that you read the whole section before trying to run what we code now. The full source code can be downloaded from the book's website, featuring inline comments. In the book, we removed the comments for brevity's sake.

Getter

The first thing we will implement is our `getter` method since it's simple enough:

```
function ResourceManager:get ( name )
    if (not self:loaded ( name )) then
        self:load ( name )
    end

    return cache[name]
end
```

This method receives a `name` parameter that is the identifier of the resource we're working with. On the first line, we call `loaded` (a method that we will define soon) to see if the resource identified by `name` was already loaded. If it was, we just need to return the cached value, but if it was not we need to load it, and that's what we do in the `if` statement. We use the internal `load` method (which we will define later as well) to take care of the loading. We will make this `load` method store the loaded object in the `cache` table. So after loading it, the only thing we have to do is return the object contained in the `cache` table indexed by `name`.

One of the auxiliary functions that we use here is `loaded`. Let's implement it since it's really easy to do so:

```
function ResourceManager:loaded ( name )
  return cache[name] ~= nil
end
```

What we do here is check whether the `cache` table indexed by the `name` parameter is not equal to `nil`. If `cache` has an object under that key, this will return `true`, and that's what we were looking for to decide whether the object represented by the `name` parameter was already loaded.

Loader

`load` and its auxiliary functions are the most important methods of this module. They'll be slightly more complex than what we've done so far since they make the magic happen. Pay special attention to this section. It's not particularly hard, but it might get confusing. Like the previous methods, this one receives just the `name` parameter that represents the asset we're loading as follows:

```
function ResourceManager:load ( name )
```

First of all, we retrieve the definition for the resource associated to `name`. We make a call to the `get` method from `ResourceDefinitions`, which we defined earlier as follows:

```
        local resourceDefinition = ResourceDefinitions:get( name )
```

If the resource definition does not exist (because we forgot to define it before), we print an error to the screen, as follows:

```
        if not resourceDefinition then
            print("ERROR: Missing resource definition for " .. name
    )
```

If the resource definition was retrieved successfully, we create a variable that will hold the resource and (pay attention) we call the correct `load` auxiliary function, depending on the asset type.

```
        else
            local resource
```

Remember the `RESOURCE_TYPE_[type]` constants that we created in the `ResourceDefinitions` module? This is the reason for their existence. Thanks to the creation of the `RESOURCE_TYPE_[type]` constants, we now know how to load the resources correctly. When we define a resource, we must include a `type` key with one of the resource types. We'll insist on this soon. What we do now is call the correct `load` method for images, tiled images, fonts, and sounds, using the value stored in `resourceDefinition.type` as follows:

```
        if (resourceDefinition.type == RESOURCE_TYPE_IMAGE) then
            resource = self:loadImage ( resourceDefinition )
        elseif (resourceDefinition.type == RESOURCE_TYPE_TILED_IMAGE) then
            resource = self:loadTiledImage ( resourceDefinition )
        elseif (resourceDefinition.type == RESOURCE_TYPE_FONT) then
            resource = self:loadFont ( resourceDefinition )
        elseif (resourceDefinition.type == RESOURCE_TYPE_SOUND) then
            resource = self:loadSound ( resourceDefinition )
        end
```

After loading the current resource, we store it in the `cache` table, in an entry specified by the `name` parameter, as follows:

```
        -- store the resource under the name on cache
        cache[name] = resource
        end
    end
```

Now, let's take a look at all of the different load methods. The expected definitions are explained before the actual functions so you have a reference when reading them.

Images

Loading images is something that we've already done, so this is going to look somewhat familiar.

In this book, we'll have two ways of defining images. Let's take a look at them:

```
{
    type = RESOURCE_TYPE_IMAGE
    fileName = "tile_back.png",
    width = 62,
    height = 62,
}
```

As you may have guessed, the `type` key is the one used in the `load` function. In this case, we need to make it of type `RESOURCE_TYPE_IMAGE`.

Here we are defining an image that has specific `width` and `height` values, and that is located at `assets/title_back.png`. Remember that we will use `ASSET_PATH` in order to avoid writing `assets/` a zillion times. That's why we're not writing it on the definition.

Another useful definition is:

```
{
    type = RESOURCE_TYPE_IMAGE
    fileName = "tile_back.png",
    coords = { -10, -10, 10, 10 }
}
```

This is handy when you want a specific rectangle inside a bigger image. You can use the `cords` attribute to define this rectangle. For example, we get a square with 20 pixel long sides centered in the image by specifying `coords = { -10, -10, 10, 10 }`.

Now, let's take a look at the actual `loadImage` method to see how this all falls into place:

```
function ResourceManager:loadImage ( definition )
    local image
```

First of all, we use the same technique of defining an empty variable that will hold our image:

```
    local filePath = ASSETS_PATH .. definition.fileName
```

We create the actual full path by appending the value of `fileName` in the definition to the value of the `ASSETS_PATH` constant. `if` checks whether the `coords` attribute is defined:

```
    if definition.coords then
    image = self:loadGfxQuad2D ( filePath, definition.coords )
```

Then, we use another auxiliary function called `loadGfxQuad2D`. This will be in charge of creating the actual image. The reason why we're using another auxiliary function is that the code used to create the image is the same for both definition styles, but the data in the definition needs to be processed differently. In this case, we just pass the coordinates of the rectangle.

```
  else
    local halfWidth = definition.width / 2
    local halfHeight = definition.height / 2
    image = self:loadGfxQuad2D(filePath, {-halfWidth, -
halfHeight, halfWidth, halfHeight} )
```

If there were no `coords` attribute, we'd assume the image is defined using `width` and `height`. So what we do is to define a rectangle that covers the whole width and height for the image. We do this by calculating `halfWidth` and `halfHeight` and then passing these values to the `loadGfxQuad2D` method. Remember the discussion about the texture coordinates in Moai SDK; this is the reason why we need to divide the dimensions by 2 and pass them as negative and positive parameters for the rectangle. This allows it to be centered on (0, 0). After loading the image, we return it so it can be stored in the cache by the `load` method:

```
    end
    return image
end
```

Now the last method we need to write is `loadGfxQuad2D`. This method is basically doing the same as what we did in the previous chapter to display an image as follows:

```
function ResourceManager:loadGfxQuad2D ( filePath, coords )
    local image = MOAIGfxQuad2D.new ()

    image:setTexture ( filePath )
    image:setRect ( unpack(cords) )

    return image
end
```

Lua's `unpack` method is a nice tool that allows you to pass a table as separate parameters. You can use it to split a table into multiple variables as well:

```
x, y = unpack ( position_table )
```

What we do here is instantiate the `MOAIGfxQuad2D` class, set the texture we defined in the previous function, and use the coordinates we constructed to set the rectangle this image will use from the original texture. Then we return it so `loadImage` can use it.

Well! That was it for images. It may look complicated at first, but it's not that complex.

The rest of the assets will be simpler than this, so if you understood this one, the rest will be a piece of cake.

Tiled images

Another useful type of asset is tiled images. In gaming, we usually use tiled images to load a single texture and use fragments of it for specific purposes (for example, animations can be achieved using this technique). This will be handy in our game *Concentration*, to define which tiles should be placed in the matrix.

The tiled image definition is as follows:

```
{
    type = RESOURCE_TYPE_TILED_IMAGE,
    fileName = 'tiles.png',
    tileMapSize = {3, 2}
}
```

This is basically the same as the normal image, but we need to add `tileMapSize` and remove `width` and `height`. This table will be unpacked as the parameters for `MOAITileDeck2D:setRect` (check it out at `http://getmoai.com/docs/class_m_o_a_i_tile_deck2_d.html#a504da8814f74038af9d453ebbc7fd5ae`). In this case, it says that the tile map has three columns and two rows, giving a total of six tiles.

`loadTiledImage` is as follows:

```
function ResourceManager:loadTiledImage ( definition )

    local tiledImage = MOAITileDeck2D.new ()

    local filePath = ASSETS_PATH .. definition.fileName

    tiledImage:setTexture ( filePath )
```

This is essentially the same as what we did with the image. We append `fileName` to `ASSETS_PATH` to get the final texture `filePath`.

```
        tiledImage:setSize ( unpack (definition.tileMapSize) )
```

The only real difference is that we unpack the `tileMapSize` attribute to pass it to `setSize`. You may use all of the possible parameters that can be passed to `setSize`; these are explained in the Moai SDK API documentation.

```
        -- return the final image
        return tiledImage

    end
```

And this is pretty much the same as for the normal image. We return `tiledImage` to be used later.

Fonts

Fonts are easier, and the definition is as follows:

```
    {
        type = RESOURCE_TYPE_FONT,
        fileName = 'myfont.ttf',
        glyphs = "0123456789",
        fontSize = 26,
        dpi = 160
    }
```

The specific attributes that we have here, besides the type and the filename, are `glyphs` (these are the characters that we will use; in this example, we only use the numbers from 0 to 9), `fontSize`, and `dpi` for font.

The `loadFont` method is as follows:

```
    function ResourceManager:loadFont ( definition )
        local font = MOAIFont.new ()
        local filePath = ASSETS_PATH .. definition.fileName
        font:loadFromTTF ( filePath, definition.glyphs,
        definition.fontSize, definition.dpi )
        return font
    end
```

The whole method is basically the same as the other loaders; the differences are that we use `MOAIFont` as the class, and we use `loadFromTTF` with the necessary parameters to initialize it.

Sounds

The definition for sounds is as follows:

```
{
  type = RESOURCE_TYPE_SOUND,
  fileName = 'sugarfree.mp3',
  loop = true,
  volume = 0.6
}
```

The specific attributes for sounds are `loop` (whether the sound should loop constantly or not) and `volume` (the initial volume for it).

Moai SDK comes with support for two different sound engines, **Untz**, an open source engine created specifically for Moai SDK, and **FMOD**, an industry standard sound library. It's worth mentioning that FMOD costs money, while Untz is free.

We're going to show you how to use Untz in this book, but you can safely code `loadSound` to use FMOD instead (take a look at `MOAIFmodExSound` and `MOAIFmodExChannel` in the Moai SDK API documentation).

```
function ResourceManager:loadSound ( definition )
    local sound = MOAIUntzSound.new ()

    local filePath = ASSETS_PATH .. definition.fileName
    sound:load ( filePath )

    sound:setVolume ( definition.volume )
    sound:setLooping ( definition.loop )

    return sound
end
```

This is basically the same logic as for the previous loaders, but we're using `setVolume` and `setLooping` to use the attributes we chose on our resource definition. We've not initialized Untz yet; this will be done later, but this code has been explained here for the completeness of `ResourceManager`.

Exercises

Putting this together should be something you can achieve now. So, take the source code of the `main.lua` file we created previously and modify it in order to make use of our `ResourceManager` and `ResourceDefinitions` entities. You'll need to include `resource_definition.lua` and `resource_manager.lua` in `main.lua` and then make good use of them. If you download the source code for this chapter from the website, `main.lua` has already been modified to work with `ResourceManager`.

Another exercise is to code a way to unload assets from `ResourceManager`. This has also been done in the code for this chapter, which you can download from the website.

> The `unload` method is pretty important since it will be needed in order to actually free the memory reserved for assets.

Another interesting topic to enhance our `ResourceManager` entity is the use of serialized fonts so font loading is faster at runtime.

Summary

In this chapter we created a simple framework to work with assets. We created ways to define images, sounds, and fonts and coded the `ResourceDefinitions` class to store this data. We wrote our `ResourceManager` class, which is in charge of loading the different assets and caching them. We also went through the creation of loading functions for these different assets and explained a way to define them. Now we're ready to start developing the gameplay for our game; how awesome is that!

www.allitebooks.com

7
Concentration Gameplay

After talking a little bit about assets and having our `ResourceManager` in place, we'll start building the *Concentration* gameplay. We'll talk about input handling, basic animations, and game states. These are the basis of gameplay implementation and will be useful from the smallest to the biggest game you develop using Moai SDK.

Grid

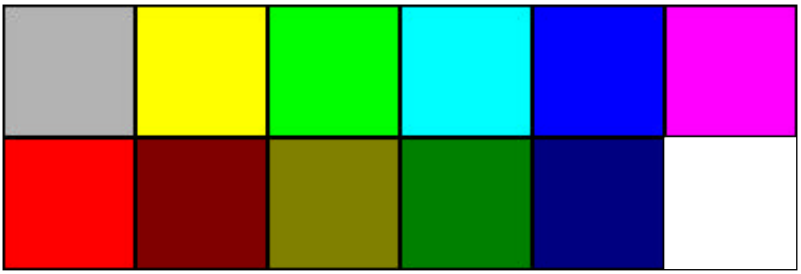
The first thing we're going to do is to display a grid of tiles that will be our playground. Just to start with, we'll display the back of the tiles.

Tilemaps

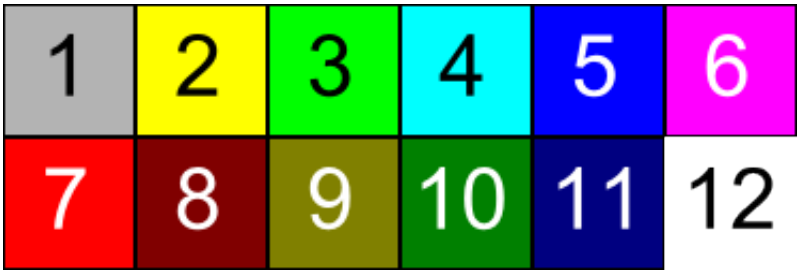
A commonly used technique in game development is the implementation of **tilesets**.

The idea is to create one image that includes more than one asset in it.

We're going to create a tileset called `tiles.png`. It consists of 12 tiles, 11 different colors (the gray one for the back of the tiles), and the "empty tile" (white). It looks somewhat like the following:



You should be aware that what we're going to do is to create a tiled image that will be indexed as follows:



This is the result of subdividing the actual image in two rows and six columns. We'll end up having twelve 62 x 62 tiles.

The implementation

Let's start with the implementation:

1. We need to create a file called `game.lua` that will hold all of the code for our gameplay.
2. Open it and let's begin coding.

```
module("Game", package.seeall)
GRID_COLS = 5
GRID_ROWS = 4
GRID_TILE_WIDTH = 62
GRID_TILE_HEIGHT = 62
BACK_TILE = 1
```

3. These constants will become useful as they define the grid dimensions (rows, columns, and tile size). The last one is used to avoid magic numbers when setting the default grid state. As we saw, the gray tile that we'll use as the back of the tiles is indexed by the number `1` (because it's the first in our tileset).

```
local resource_definitions = {

  tiles = {
    type = RESOURCE_TYPE_TILED_IMAGE,
    fileName = 'tiles.png',
    tileMapSize = {6, 2},
  },

}
```

4. This should be familiar to you now. To define our tileset, we'll use the tiled image called `tiles.png` that we created earlier, which has six columns and two rows.

```
function Game:start ()
  -- Do the initial setup
  self:initialize ()

end
```

5. We're creating a method called `Game:start()` that will take care of initializing everything and controlling the game loop. This will be called from `main.lua` later. For now, we will only display our grid on initialization. So what we'll do here is call a function named `initialize()` that will take care of all the actual initialization. We'll be adding more code to this method later.
6. The `initialize ()` function is as follows:

```
function Game:initialize ()

  self.layer = MOAILayer2D.new ()
  self.layer:setViewport ( viewport )
```

   1. We make a layer that will handle all of the rendering from the gameplay (including our grid) and configure the viewport on it.

      ```
      MOAIRenderMgr.setRenderTable ({ self.layer })
      ```

   2. Then we set the render table using that layer.

      ```
      ResourceDefinitions:setDefinitions (resource_definitions)
      ```

   3. The next step is to use `ResourceDefinitions` with a helper method called `setDefinitions ()`.

      ```
      self:initializeTiles ()
      ```

   4. After loading the resource definitions, we call another auxiliary function that initializes the tiles.

      ```
      end
      ```

The idea behind `setDefinitions ()` is to take a table of definitions and load all of them at once, instead of one by one. It was not created in the previous chapter, but it should be pretty simple to implement. Just iterate through the parameter table and call `setDefinitions ()` on each entry. Please go ahead and try to implement it by yourself. Remember, you can always download the book's code, and the method will be there.

7. Now we need to take care of `initializeTiles ()`:

```
function Game:initializeTiles ()

  local grid = MOAIGrid.new ()
  grid:setSize ( GRID_COLS, GRID_ROWS,
                 GRID_TILE_WIDTH, GRID_TILE_HEIGHT )
```

   1. First of all, we create the grid using the `MOAIGrid` class. Then we set it up. We use the grid and tile dimensions we defined earlier to configure our new `MOAIGrid`.

```
grid:setRow ( 1, BACK_TILE, BACK_TILE,
                 BACK_TILE, BACK_TILE, BACK_TILE )
grid:setRow ( 2, BACK_TILE, BACK_TILE,
                 BACK_TILE, BACK_TILE, BACK_TILE )
grid:setRow ( 3, BACK_TILE, BACK_TILE,
                 BACK_TILE, BACK_TILE, BACK_TILE )
grid:setRow ( 4, BACK_TILE, BACK_TILE,
                 BACK_TILE, BACK_TILE, BACK_TILE ) )
```

   2. Now we set up the actual grid. We call the `setRow()` method with the row number as the first parameter. (Start from `1`; remember, we're using Lua!) The following parameters are the values of the tiles we want to show for each column. We will show the back tile across all the grids as an initial state for our game; that's why, we pass `BACK_TILE` as the tile number.

Go ahead and play a little bit, changing the values for these parameters. (Not the first one though; that's the row!) Put some numbers between `2` and `11`; you should expect to see some color on the gray sea.

```
self.tiles = {}
self.tiles.grid = grid
self.tiles.tileset = ResourceManager:get ('tiles')

self.tiles.prop = MOAIProp2D.new ()
self.tiles.prop:setDeck ( self.tiles.tileset )
self.tiles.prop:setGrid ( self.tiles.grid )
```

   3. Then we create a table called `tiles` and store the tileset in there, as well as a new Prop. The usage of a Prop is basically the same as we did earlier; the only difference is that we now use `setGrid ()` in order to make the Prop follow the size and configuration of that grid.

```
self.tiles.prop:setLoc ( - GRID_COLS/2 * GRID_TILE_WIDTH,
                         - GRID_ROWS/2 * GRID_TILE_HEIGHT )
```

4. Since grids don't center over their elements (they just draw them from offsets), we need to center them ourselves. In this case, `(0,0)` is at the bottom-left corner of the Prop (instead of the middle). We need to offset it, and that's what we do by calling the `setLoc()` method; we move it by half of its columns times the tile's width (in the *x* axis), and by half of its rows times the tile's height (in the *y* axis).

Moai SDK has a family of methods to deal with position, rotation, and scale. There are three families, namely the `set*` methods (that force a specific value), the `move*` methods (that generate animations applying a delta over time), and the `seek*` methods (that generate animations by going from the actual value to the specified value in a certain amount of time). To modify position you use `Loc`, for rotation `Rot`, and for scale `Scl`. So, for example, if you wanted to rotate the `tiles` Prop by 30 degrees within 10 seconds, you'd call `self.tiles.prop:moveRot (30, 10)`.

There's some more to these methods (like the curve they use to interpolate the values); you should check them out at `http://getmoai.com/docs/class_m_o_a_i_transform2_d.html`.

```
    self.layer:insertProp ( self.tiles.prop )
  end
```

5. We add the Prop to the layer and we're done.

8. The only thing remaining is to call this from `main.lua`. Below the viewport definition add the following code:

```
function mainLoop ()
  Game:start ()
end
gameThread = MOAICoroutine.new ()
gameThread:run ( mainLoop )
```

We will run our game in a separate Lua coroutine; this is done in order to allow it to have a game loop that is detached from the main coroutine flow. This will be handy for processing input.

That should be it; if you run the project now, you should see the grid with all 20 back tiles.

Input

The next step in our journey is to add input handling. We have to create a file called `input_manager.lua`; it will be in charge of listening to input events and will be queried by our game when needed. We will create it so that it will be usable with both mouse devices and touchscreens.

Generally speaking, all hosts are supposed to implement their own input schema. We won't see this in detail in this book, but so you understand where this comes from, take a look at the hosts that are shipped with Moai SDK.

Search for all the references to the `AKUReserveInputDevice*` and `AKUSetInputDevice*` method calls in `moai-sdk/hosts/src/GlutHost.cpp`.

We'll now inspect only the most important methods; please take a look at the full code in order to understand how everything fits together.

```
function InputManager:initialize ()

  if MOAIInputMgr.device.pointer then

  local pointerCallback = function ( x, y )
    previousX, previousY = pointerX, pointerY
    pointerX, pointerY = x, y

    if touchCallbackFunc then
      touchCallbackFunc ( MOAITouchSensor.TOUCH_MOVE, 1,
        pointerX, pointerY, 1 )
    end
  end

  MOAIInputMgr.device.pointer:setCallback ( pointerCallback )

end
```

As you can see, this method checks if we have a pointer, and if we do, it creates a callback function (`pointerCallback`) that is passed to that pointer in order to update the `x` and `y` positions. It also talks to the touch sensor (`MOAITouchSensor`) in order to do the same in case we're not using a mouse but a touch-screen instead.

Besides this, another important method is `isDown ()`, which is used to know if we're clicking or touching the screen.

```
function InputManager:isDown ()

  if MOAIInputMgr.device.touch then
```

```
    return MOAIInputMgr.device.touch:isDown ()

  elseif MOAIInputMgr.device.pointer then

    return (
      MOAIInputMgr.device.mouseLeft:isDown ()
    )
  end
end
```

This method is querying both the mouse and the touch sensor asking if we're clicking or touching respectively. We'll use this in our game loop to identify input. Remember to include this file in `main.lua` after the `require 'resource_manager'` statement.

Gameplay

Now let's take a look at what we need in order to implement the gameplay. First of all, we need to create all of the structures (and initialize them) for the game state. We'll be using another grid, this time to store a randomized distribution of numbers.

Initialization

1. Add a call at the bottom of the `Game:initialize ()` method to an auxiliary function.

   ```
   self:restartGamePlay ()
   ```

2. Add the auxiliary function as well.

   ```
   function Game:restartGamePlay ()
     self.distributionGrid = MOAIGrid.new ()
     self.distributionGrid:setSize (GRID_COLS, GRID_ROWS)
   ```

   1. First, we create a grid that will hold the distribution of our colors. This grid also has a dimension of 5 x 4. Note that we don't need to set the tile size now as this grid won't be rendered.

      ```
      local tiles = {
        2, 2, 3, 3, 4, 4, 5, 5, 6, 6,
        7, 7, 8, 8, 9, 9, 10, 10, 11, 11
      }
      ```

2. This table is just a temporary array of tiles; as you see, we start from `2` (`1` is the back tile, remember?) and repeat each number twice. These numbers represent the color (or the tile offset) on our tileset.

```
for x=1, GRID_COLS, 1 do
  for y=1, GRID_ROWS, 1 do
    local random =  math.random ( 1, #tiles )
    local value = tiles[random]
    table.remove ( tiles, random )
    self.distributionGrid:setTile ( x, y, value )
  end
end
```

3. The next thing we do is to iterate over all of the grid cells, assign one of the values to the `tiles` table at random, and remove it from the table so it doesn't get selected more than once. In this way, we fill our `distributionGrid` with a randomized order of the color pairs. Nice.

```
  self.selectedCells = { nil, nil }
end
```

4. The next thing we need is a place to store the cells that were selected in the current turn. We'll use a table called `selectedCells`, and it will start displaying two `nil` cells because we haven't selected anything yet.

And that's it for the gameplay initialization part.

Input processing

Now we need to process input from the user.

1. We need to append some code to the `Game:start ()` method (below the call to `Game:initialize ()`):

```
self.wasClicking = false
```

   1. We want to know if in the previous simulation step the user was clicking or tapping the screen. We'll store that information in `wasClicking`.

```
while (true) do
  self:processInput ()
  coroutine.yield ()
end
```

2. Then we add an infinite loop (beware of algorithms!) that *processes input* and uses the magical method `coroutine.yield()` that delegates the flow to the main routine, allowing `MOAISim` to do another simulation step. In this way, we avoid having an endless loop that freezes the game. Instead, we do all the rendering, Action Tree and Node Graph processing; and when all of that is done, it comes back to our coroutine to process the input once more.

If you expect your game to update the screen (render) and it's not doing it, it's likely that you've forgotten to run `coroutine.yield()` to allow rendering before continuing with your code. Think about the loading screens for example.

2. Now the next method we'll take care of is the one that actually processes the input.

```
function Game:processInput ()

  if InputManager:isDown () and not self.wasClicking then
```

   1. If the player is clicking now and was not clicking in the previous simulation step (to avoid selecting multiple times when holding the mouse button), we want to select a tile.

      ```
      x, y = InputManager:position ()
      ```

   2. The way we choose the tile is a pretty neat trick. We gather the mouse position that is represented in the window coordinate system.

      ```
      worldX, worldY = self.layer:wndToWorld ( x, y )
      modelX, modelY = self.tiles.prop:worldToModel ( worldX, worldY )
      ```

   3. We need to transform them into model coordinates to do our trick by first converting them to world coordinates (through the layer). After these conversions, what we get in `modelX` and `modelY` is the offset in pixels from the bottom-left corner of our grid.

      ```
      cellColumn = math.floor ( modelX / GRID_TILE_WIDTH ) + 1
      cellRow = math.floor ( modelY / GRID_TILE_HEIGHT ) + 1
      ```


www.allitebooks.com

4. If we divide `modelX` and `modelY` by the dimensions of the cells, we'll get the cell column and row indices. Nice *mathemagics*. Now it's just a matter of calling an auxiliary function that will take care of the consequences of that click:

```
    self:choseCell ( cellColumn, cellRow )
  end

  self.wasClicking = InputManager:isDown ()

end
```

5. Oh! And don't forget to update `wasClicking`. We need to store the value of `InputManager:isDown ()` on it because we'll need it in the next simulation step.

Choosing a cell

Now we have to implement the following cell-choosing method:

```
function Game:choseCell ( column, row )

  if not self.selectedCells[1] then
```

1. If no cell was selected previously (the first one is nil), we have to select it.

```
    if not self:alreadySelectedTile ( column, row ) then
```

2. We always need to check that the cell we're clicking has not been removed from the game.

```
      self.selectedCells[1] = {column, row}
```

3. We put the selected cell's column and row in the `selectedCells` table.

```
      self:swapTile ( column, row )
```

4. Then we swap it to reveal its color.

```
    end
  else
```

5. We continue with the case if a tile was selected previously.

```
    if (self.selectedCells[1][1] == column) and (self.
selectedCells[1][2] == row) then
      self.selectedCells[2] = {column, row}
      self:resetTiles ()
```

6. If the player clicked the same tile, what we want is to swap it back. Set the same tile as the second selected cell (`selectedCells[2]`) and reset them.

```
    else

      if not self:alreadySelectedTile ( column, row ) then
```

7. Remember to check that the cell we're clicking has not been removed from the game.

```
        self.selectedCells[2] = {column, row}
        self:swapTile ( column, row )
```

8. The same logic is applied here to select the cell and swap it. (Oh no! code duplication!)
9. Now the core of the gameplay:

```
        local value1 = self.distributionGrid:getTile (
                         unpack(self.selectedCells[1]) )

        local value2 = self.distributionGrid:getTile (
                         unpack(self.selectedCells[2]) )
```

10. We gather both values and store them in `value1` and `value2` respectively. These variables contain the color of the tiles that the user selected.

```
        if (value1 == value2) then
          self:removeTiles ()
```

11. If the colors are the same, we remove the tiles.

```
        else
          self:resetTiles ()
```

12. If they differ, we swap them back. And that's it.

```
        end
      end
    end
  end
end
```

After a nice tail of `end` sentences, we're ready to implement the auxiliary methods.

Tile swapping

To swap our tiles, we'll create the following `swapTile ()` method:

```
function Game:swapTile ( column, row )
  local value = self.distributionGrid:getTile ( column, row )
  self.tiles.grid:setTile ( column, row, value )
end
```

What we do here is retrieve the color value from our `distributrionGrid` and set it into the rendered grid. This is really magical; don't you love it?

The result is that the tile changes its color according to the distribution on our `distributionGrid`.

Every distribution is randomized, so this will be different every time you play.

Resetting tiles

1. We need two more constants at the top of `game.lua` for the next methods (we will use these constants to avoid magic numbers):

   ```
   EMPTY_TILE = 12
   ```

   `EMPTY_TILE` is the white space that we have after all the tiles; we'll use it to remove the tiles that are not in the game. That's the index `12`.

   ```
   DELAY_TIME = 0.5
   ```

   `DELAY_TIME` is a time in seconds that we will use to defer the swapping back of the tiles, in order to allow the user to see what's the second color he has chosen. (In fact, after finishing the chapter, change it to zero and see what happens).

2. And now, we create our `resetTiles ()` method:

   ```
   function Game:resetTiles ()
     sleepCoroutine(DELAY_TIME)
   ```

   1. We freeze the coroutine during `DELAY_TIME` seconds. We'll implement `sleepCoroutine` in the following code snippet:

      ```
      self.tiles.grid:setTile (
        self.selectedCells[1][1],
        self.selectedCells[1][2], BACK_TILE )

      self.tiles.grid:setTile (
        self.selectedCells[2][1],
        self.selectedCells[2][2], BACK_TILE )
      ```

2. This one may seem tricky, but what we're doing is telling our rendering grid that the cells in `selectedCells` should render `BACK_TILE`. We use the coordinates of each selected cell and pass them to `setTile` as the first two parameters. The third one is the color, so we use `BACK_TILE`.

```
  self.selectedCells = {}
```

3. We clear the selected cells to start over.

```
end
```

That's it. Now we need to take care of removing the tiles.

Removing tiles

The following method will be used to remove the tiles that are equal and to swap the user correctly.

```
function Game:removeTiles ()
  sleepCoroutine ( DELAY_TIME )

  self.tiles.grid:setTile ( self.selectedCells[1][1], self.
selectedCells[1][2], EMPTY_TILE )
  self.tiles.grid:setTile ( self.selectedCells[2][1], self.
selectedCells[2][2], EMPTY_TILE )
```

1. This is pretty similar to the what we do when resetting tiles. The only difference is that we use the `EMPTY_TILE` constant here to make the tile disappear.

```
  self.selectedCells = {}
```

2. We clear the selected cells to start over.

```
end
```

Other auxiliary methods

There are two more auxiliary methods pending; one to see if a tile was already removed from the game and another to make the coroutine sleep. So here they are:

```
function Game:alreadySelectedTile ( column, row )
  return self.tiles.grid:getTile ( column, row ) == EMPTY_TILE
end
```

1. This method is used to identify if we removed a given tile or not. Basically we compare it's color value to `EMPTY_TILE`. If it's equal, then that tile was removed previously.

2. Now for the strange (and extremely useful) one:

```
function sleepCoroutine (time)
  local timer = MOAITimer.new ()
  timer:setSpan ( time )
  timer:start()
  MOAICoroutine.blockOnAction ( timer )
end
```

In this method, what we do is freeze the coroutine for a given time. We use `MOAITimer` to achieve this. The magic here is created by the fact that the timer inherits from `MOAIAction`, and as a consequence, we can use the built-in `MOAICoroutine.blockOnAction()` method, which basically waits until the action is finished, to continue execution on the current thread.

This is an important method, as you will see. It's used to create animation sequences in Moai SDK, when you have to wait for an action to finish before the other one starts (for example, a translation and a rotation done in sequence instead of in parallel).

Well, the game is pretty much ready for play. You'll obviously need to add a couple of things to this for it to be a ready-to-ship game (for example, a winning condition, a nice menu, a ticking clock, and so on), but the gameplay is there, and that's what we were looking for.

Summary

In this chapter, we learned how to use a grid to display tiles on the screen and also to represent gameplay data. We learned the basics of input handling and how to implement the actual gameplay of *Concentration*. We also had our first approach to how Moai SDK uses coroutines. Now it's a good time to take some rest since the next chapters are juicy; we'll build a platformer prototype that uses Box2D for physics!

8
Let's Build a Platformer!

Now that we're familiar with handling input and assets, why don't we build a platformer prototype? This should be a nice challenge that will introduce us to some more concepts and ways to work with Moai SDK.

In order to build this game, we'll copy the full content of the previous chapter's code and clean everything from `game.lua` that is specific to the *Concentration* game.

- Remove all the constants on the top of the file, the definition for the tile (but keep `resource_definitions` since we're going to use it).
- Remove `self.wasClicking = false` from the `Game:start ()` method.
- Remove the calls to `initializeTiles` and `restartGamePlay` from `Game:initialize ()`.
- Remove the methods `Game:initializeTiles()`, `Game:restartGamePlay ()`, `Game:choseCell ()`, `Game:alreadySelectedTile()`, `Game:swapTile ()`, `Game:resetTiles ()`, and `Game:removeTiles()`.

That should be it; now you should be good to go.

Background

We're going to insert a background to our screen:

1. The first thing we're going to do is display a background. This should be pretty easy for you right now. A good idea will be searching `www.opengameart.org` for a suitable background. That's what has been done in the code for this chapter, and you'll find a background in there.

2. Let's add the definition. If you're using an image other than the one provided in the book's code, just go ahead and change the dimensions and filename (this code should go inside `resources_definition`):

```
background = {
  type = RESOURCE_TYPE_IMAGE,
  fileName = 'background/wizardtower.png',
  width = 1000, height = 700
},
```

3. Now, let's add a line at the bottom of the `initialize ()` method to load the background:

```
self:loadBackground ()
```

4. And now let's define that method:

```
function Game:loadBackground()

  self.background = {}
  self.background.deck = ResourceManager:get('background')

  local prop = MOAIProp2D.new ()
  prop:setDeck (self.background.deck)
  prop:setLoc (0, 0)

  self.background.prop = prop

  self.layer:insertProp (prop)

end
```

This is nothing new, right? It's the same thing we did to load an image in *Chapter 5, Showing Images on the Screen*.

Now, let's play with the camera and scroll it.

Take into account the platform you're targeting when loading backgrounds.

The maximum texture size differs according to the **Shader Model**. You will be safe if you're using textures that are 1024 x 1024 or smaller; if you find yourself having textures that don't show up on the screen, double-check this. Remember that you can always split the textures into smaller ones and load several of them in different Props. We ran into this problem with the second scene in *The Insulines*; it wasn't working on some computers because it was huge, so we had to split it.

Camera and scrolling

If you're planning to create games in 2D, an important thing to master is how to use a camera, and that's what we'll do now.

Enter this code at the beginning of `Game:initialize ()` in `game.lua`:

```
self.camera = MOAICamera2D.new ()
```

1. What we're doing here is creating a new camera using the built-in `MOAICamera2D` class.
2. Next, we need to assign that camera to each layer. Below the viewport assignment `(self.layer:setViewport ( viewport ))`, add the following line of code:

   ```
   self.layer:setCamera ( self.camera )
   ```

   This binds the layer to that camera. And that's it; when you move the camera, you'll see that the viewport will follow, and if you scale the camera up and down, it will zoom in and zoom out accordingly. We'll write a few lines to demonstrate this.
3. Inside `Game:processInput ()` write the following code:

   ```
   local x, y = InputManager:deltaPosition ()
   ```

   We are using a method from `InputManager` that returns the difference between the current mouse position and the previous one.

   ```
   self.camera:moveLoc ( x, 0, 0.5, MOAIEaseType.LINEAR )
   ```

We move the camera using the `moveLoc ()` method and passing the delta in `x`, nothing in `y`. The third parameter tells `moveLoc ()` to take half-a-second to do the movement. The third parameter is used to define the curve of the animation, in this case, we're making it linear, so it will make a movement that is constant in speed. We'll go more into this later.

`MOAIEaseType` defines a bunch of useful curves for our animations; you can check them out at `http://getmoai.com/docs/class_m_o_a_i_ease_type.html`.

If you run the game now, you'll see that when you move the mouse, the background scrolls to the left and right. That's a consequence of the camera being moved.

So, instead of moving all of the objects on screen, you just move the camera when you want to scroll.

What we can do later is scroll the camera to follow our character's movements.

Parallax

Parallax is an effect that's often used (and abused) in games because it creates a feeling of depth in 2D scenes. The main idea is to create layers of images, which are at different distances, which move at different speeds when scrolling. This mimics what happens in reality with objects that are at different distances from a viewer (the objects far away move slower than the near ones).

We can achieve this by splitting the background into different images (each one representing a different plane).

Here we see the old background split into three layers:

- A cloudy background
- A tower that is far away
- Some hills that are nearer

Now, what we need to do is make them move at different speeds.

Setting up the layers

In Moai SDK, you can implement this using a `MOAILayer2D` method called `setParallax ()`.

Let's take a look at it.

1. As we saw, we'll need a couple of layers to do this. We'll replace our current layer with a table of layers. On `Game:initialize ()`, replace the lines for the layer initialization with a call to `self:setupLayers ()`.

2. Then we create the actual method:

```
function Game:setupLayers ()

  self.layers = {}
  self.layers.background = MOAILayer2D.new ()
  self.layers.farAway = MOAILayer2D.new ()
  self.layers.main = MOAILayer2D.new ()
```

   1. We'll create a layer per depth plane that we want. We'll use three planes in this example: a background, a far away plane, and a main plane. (Our character and all of the game objects should go inside the main plane)

```
 for key, layer in pairs ( self.layers ) do
    layer:setViewport ( viewport )
    layer:setCamera ( self.camera )
  end
```

   2. Then we iterate through all of the layers and assign the viewport and the camera correctly.

```
  local renderTable = {
    self.layers.background,
    self.layers.farAway,
    self.layers.main
  }
```

   3. Now, we create a render table with the layers in order.

```
  MOAIRenderMgr.setRenderTable(renderTable)
end
```

   4. We'll then make it active.

Now we have our layers ready to play with parallax.

Adding images to the different distances

Now we'll show images on each layer for you to see how cool this is:

```
local resource_definitions = {

  background = {
    type = RESOURCE_TYPE_IMAGE,
    fileName = 'background/background_parallax.png',
    width = 1500, height = 197,
  },
```

```
  farAway = {
    type = RESOURCE_TYPE_IMAGE,
    fileName = 'background/far_away_parallax.png',
    width = 625, height = 205,
  },

  main = {
    type = RESOURCE_TYPE_IMAGE,
    fileName = 'background/main_parallax.png',
    width = 975, height = 171,
  },

}
```

3. These are the three resource definitions we'll need: one for each layer. The images can be found in the source code for this chapter.

   There is some more information that we'll need, but instead of adding it into the `resource_definitions` table, we'll create another table. Even if the objects and definitions look the same, there is a slight conceptual difference between them, so it is better to keep them separated.

4. Write the following code below the `resource_definitions` table:

```
local background_objects = {

  background = {
    position = { 0, 70 },
    parallax = { 0.05, 0.05 }
  },

  farAway = {
    position = { 0, 50 },
    parallax = { 0.1, 0.1 }
  },

  main = {
    position = { 0, -75 },
    parallax = { 1, 1 }
  },

}
```

   What we're doing here is defining the position and the parallax information for each object we're creating. Well, strictly, the parallax is for the layer that contains the object.

5. Now, we'll rewrite `Game:loadBackground ()` to look like the following:

```
function Game:loadBackground ()

  self.background = {}

  for name, attributes in pairs(background_objects) do
```

   1. We iterate through all the things we defined in `background_objects`:

```
      local b = {}
      b.deck = ResourceManager:get ( name )
```

   2. We load the deck using the keys from the `background_objects` table.

```
      b.prop = MOAIProp2D.new ()
      b.prop:setDeck ( b.deck )
      b.prop:setLoc ( unpack(attributes.position) )
```

   3. Here we use the position attribute we defined in `background_objects`:

```
      self.layers[name]:insertProp ( b.prop )
self.layers[name]:setParallax ( unpack(attributes.parallax)
)
```

   4. And here we make use of `setParallax` and use the parallax attribute we defined in background objects:

```
      self.background[name] = b
    end

end
```

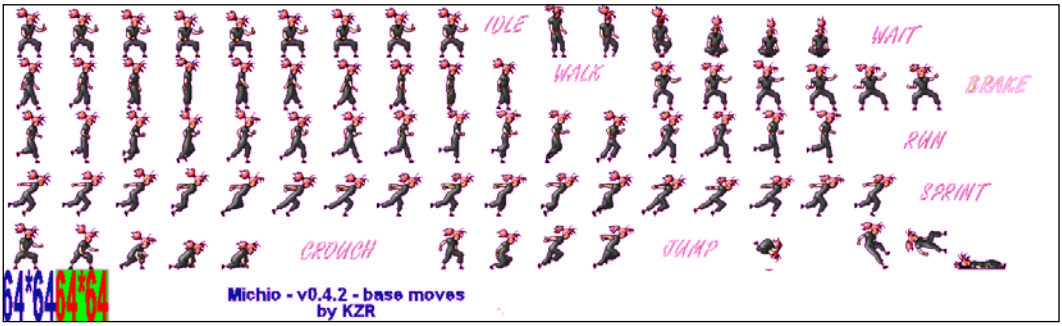
And that's it, we're now good to go. Run the game and move the mouse. You should now see the three different layers moving at different speeds.

Now we are ready to add the main character, so let's jump straight to that!

Main character

In order to create our character, we'll need a sprite sheet with animations for walking, running, and jumping.

It should look something like the following image:



6. First of all, we'll create a resource definition as follows (put this below the background resource definition):

```
character = {
  type = RESOURCE_TYPE_TILED_IMAGE,
  fileName = 'character/character.png',
  tileMapSize = {20, 6},
  width = 64, height = 64,
}
```

This is pretty much the same as we used for our *Concentration* game, but note that we added `width` and `height`. That's the size of the tile. In the *Concentration* game, we defined the size of the tile in the grid, but now there is no grid, so we need to define it here.

1. In order to handle that definition, we need to add some lines to `resource_manager.lua`, inside `loadTiledImage`, just below the call to `setSize()`:

```
if definition.width and definition.height then
  local half_width = definition.width / 2
  local half_height = definition.height / 2
  tiledImage:setRect ( -half_width, -half_height, half_width,
half_height )
end
```

The preceding code is similar to that in `loadImage`; the difference is that we divide `width` and `height` by two and use the resulting values as the rectangle's dimensions.

2. Let's create a file called `character.lua` to start creating our character (don't forget to `require` that file in `game.lua`) and add the following code to it:

```
module ( "Character", package.seeall )

local character_object = {
  position = { 0, 0 },
}
This table will be used to setup some initialization parameters.
function Character:initialize (layer)
  self.deck = ResourceManager:get ( 'character' )
  self.prop = MOAIProp2D.new ()
  self.prop:setDeck ( self.deck )
  self.prop:setLoc ( unpack(character_object.position) )
  layer:insertProp ( self.prop )
end
```

This is basically the same as what we've been doing in order to load images on screen; the important part here is that initialize receives a layer as a parameter and adds the Prop to that layer. Can you guess why?

Here is the reason: put the following line at the bottom of `Game:initialize ()` in `game.lua` (just below the call to `loadBackground`):

```
Character:initialize ( self.layers.main )
```

And voilà, we have our character on screen. Loving it? I think it's pretty static. Let's add some animations.

Animations

Now we are going to work on the animations. We'll introduce three new classes in this section: `MOAIDeckRemapper`, `MOAIAnimCurve`, and `MOAIAnim`. Let's see how these interact.

1. First of all, on the `character_object` table on `character.lua`, let's add the following (below position):

```
animations = {
    idle = {
      startFrame = 1,
      frameCount = 9,
      time = 0.1,
      mode = MOAITimer.LOOP
    },
```

```
    run = {
      startFrame = 41,
      frameCount = 16,
      time = 0.03,
      mode = MOAITimer.LOOP
    },

    jump = {
      startFrame = 89,
      frameCount = 3,
      time = 0.1,
      mode = MOAITimer.NORMAL
    },
  }
```

This is the definition for the three animations we're going to use. Take a look at it and see that we define a start frame (the tileset index where the animation starts), the frame count (the amount of frames for the animation; these frames need to be sequentially placed in the tileset, as per the preceding example), the time between each frame, and the animation mode.

Similar to `MOAIEaseType`, `MOAITimer` has a bunch of different animation modes. You can check them out at `http://getmoai.com/docs/class_m_o_a_i_timer.html`.

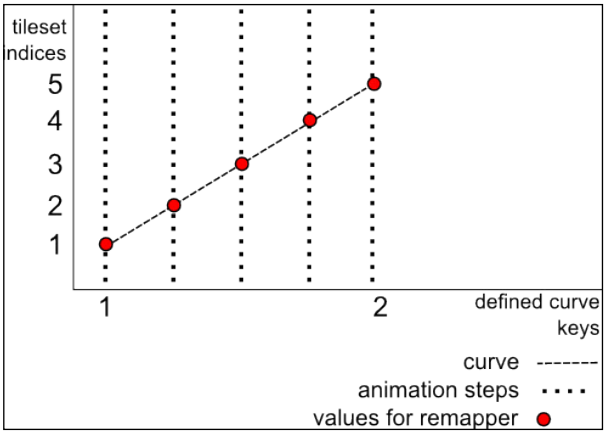
1. Then, we need to add a `MOAIDeckRemapper`. `MOAIDeckRemapper` is used in combination with an animation curve in order to tell the Prop which tileset index should be displayed in each animation step. Enter the following code in `Character:initialize()`:

   ```
   self.remapper = MOAIDeckRemapper.new ()
   self.remapper:reserve ( 1 )
   ```

   1. We'll remap just one index (the character animation frame), so we reserve just one remapper index.

      ```
      self.prop:setRemapper ( self.remapper )
      ```

   2. Then we assign the remapper to the Prop.



For the animation to work, we need to create a curve. It will be used to tell our `MOAIDeckRemapper` instance which tileset index should be used at a given animation step.

The remapper will be in charge of changing the current Prop's index to the correct one.

The interesting thing is that these curves are defined using keys that set the value for a given point in the animation. These values are then interpolated using a `MOAIEaseType` curve.

This is really nice, since we don't have to specify each point in our animation, just two of them and the rest are interpolated. Let's see how this works.

2. Let's create a method that will add an animation and review it step by step.

```
function Character:addAnimation ( name, startFrame, frameCount,
time, mode )
  local curve = MOAIAnimCurve.new ()
```

   1. First, we create an animation curve. These curves are used to generate a specific behavior on how to move through the frames. We create a series of points that will create a curve (interpolating values). The animation will query the curve at a specific point of time and use its value.

```
curve:reserveKeys ( 2 )
```

2. A **key** is a value that we use to define the curve. We'll need two keys here (the start and the end of the curve), but you might want more than one (for example, if some part of the animation needs to run faster than the others or stuff like that).

```
curve:setKey ( 1, 0, startFrame, MOAIEaseType.LINEAR )
```

3. Using `setKey ()` we define a key:

The first parameter is the key number (starting at one). The second is the time at which that key should happen; in this case, since it's the first key for our animation, the time is zero. Then, we pass the value of that key; in our case, it will be our `startFrame`, and then `MOAIEaseType`. Ease types are a way to tell animations to use different curves. In this case, we're using a linear curve.

```
curve:setKey ( 2, time * frameCount,
  startFrame + frameCount, MOAIEaseType.LINEAR )
```

4. The second key will be the last one. For the second parameter (`time`), we multiply the time between frames by the frame count, should be the total time for the animation. After that, we tell it to stop at `frameCount` frames after `startFrame` and, again, we use linear interpolation.

When creating our animation, this will create a linear function from `startFrame` at time `0` to `startFrame + frameCount` at the end of the curve. Just what we want for our animation.

```
local anim = MOAIAnim:new ()
anim:reserveLinks (1)
anim:setLink ( 1, curve, self.remapper, 1 )
```

5. Next, we create our animation, reserve one link that will be used to connect the curve to the remapper, and then we connect them.

What will happen here is that when we start the animation, it will iterate through the curve passing the result of the curve to the remapper. When that happens, the remapper will change the image displayed by the Prop. The one at the end of the call is the index of the remapper we'll be using; since we only reserved one, we pass one there.

```
anim:setMode ( mode )
```

6. Here we set the animation mode we defined.

```
self.animations[name] = anim

end
```

7. To finish this, we store the animation under the name we passed as a parameter.

The way this works is not trivial, so it may take a while for you to process it. You should play around with the definition and parameters to get an idea of how the `MOAIAnim` + `MOAIAnimCurve` + `MOAIDeckRemapper` workflow interacts.

We'll define some auxiliary methods to work with these animations:

```
function Character:getAnimation ( name )
  return self.animations[name]
end
```

3. `getAnimation` returns the animation that is indexed by name.

```
function Character:stopCurrentAnimation ()

  if self.currentAnimation then
    self.currentAnimation:stop ()
  end

end
```

4. In this method, we stop the animation that is currently running if it exists.
5. The following method is used to start an animation. Let's take a look at it.

```
function Character:startAnimation ( name )

  self:stopCurrentAnimation ()
```

   1. First, we stop the current animation.
   2. Stopping the animation is really important, because a running animation consumes some memory, and if we don't stop it, we'll end up having tons of animations running all the time and consuming a lot of memory. We figured this out while developing *The Insulines* as well.

      ```
      self.currentAnimation = self:getAnimation ( name )
      ```

   3. Then we get the animation indexed by name.

      ```
      self.currentAnimation:start ()
      ```

   4. And then we start it.

      ```
      return self.currentAnimation
      end
      ```

5. We then return it, in case we want to do something with it later (such as blocking the coroutine to chain multiple animations, remember?).

6. The last thing we need to do is parse all of the animations we defined inside `character_object` and create the necessary animations. To do this, go to `Character:initialize`, and at the bottom, include the following code:

```
self.animations = {}

for name, def in pairs ( character_object.animations ) do
  self:addAnimation ( name, def.startFrame, def.frameCount,
    def.time, def.mode )
end
```

Here we are iterating through all of the defined animations and are using the attributes as parameters for our `addAnimation ()` method.

The result will be that all the animations will be inside `Character.animations` and we'll be able to access them through the methods created earlier (`startAnimation` and `stopAnimation`). To see if everything works fine, add the following line:

```
self:startAnimation ( 'run' )
```

This should start the running animation. You will want to experiment a little bit with this by changing the parameters in the definitions and calling different animations.

Summary

In this chapter, we started to implement the initial structure for our platformer prototype. We created a camera and learned how to move it. We also played with layers and parallax. Then we added our character and learned about how to create animations from a tiled image.

In the next chapter, we'll introduce some physics to the prototype using Box2D, which is already shipped with Moai SDK.

9
Real-world Physics with Box2D

If you've already worked on games before, you should be familiar with Box2D. It's a multiplatform physics library that is broadly used across the gaming community. You'll find it really easy to work with it and get nice-looking physics pretty quickly, but this doesn't mean you have to create another *Angry Birds* clone, hint hint.

Creating the world

The first step is to create the world that will be in charge of the physics simulation by following the given steps:

1. Create a file called `physics_manager.lua` and `require` it from `game.lua`.
2. Now let's build the `PhysicsManager` module.

   ```
   module ( "PhysicsManager", package.seeall )
   function PhysicsManager:initialize ( layer )
   ```

   1. The first step is to create the `MOAIBox2DWorld` object. This will take care of talking to Box2D from Moai SDK.

      ```
      self.world = MOAIBox2DWorld.new ()
      ```

   2. Then, we need to set up our scale. What we're doing here is saying that one meter on the simulation is equivalent to 38 points in our world coordinates. Since we're using a main character sprite that is 64 points tall, we can say that our character could be about 1.67 meters tall. So, 65 / 1.67 is about 38. That's the calculation needed in order to find that scale.

      ```
      self.world:setUnitsToMeters ( 1/38 )
      ```

> Box2D works better on simulations that contain bodies that go from 0.1 to 10 meters, so take that into account when setting this scale.

3. Here we're setting the gravity. The parameter is how many meters per second in `x` and `y` our objects get affected. So, for this example, we're using a gravity that is similar to the one on Earth (which is -9.8 mts / (s * s), so we can safely use -10).

```
self.world:setGravity ( 0, -10 )
```

4. And finally we start the simulation:

```
self.world:start ()
```

5. Normally, we don't want to see the objects that Box2D creates on our screen; we just want them to be connected to our own rendered objects and make them behave according to the simulation. But, for debugging purposes, having the physics objects displayed on screen can be pretty handy. If you pass a layer to the `initialize` method, it will use it as a debug layer to display the objects you create on Box2D. You'll see the actual shapes of the physics objects, and their colors will change when they collide with each other.

```
    if layer then
        layer:setBox2DWorld ( self.world )
    end
end
```

That's it. We have our world working! Now it's time to have a little talk about bodies.

Box2D body types

The whole point of using a physics engine is to create a world where you put objects (bodies) that behave according to physics laws. The term `body` is highly based on physics literature, where all the objects that interact are called bodies.

There are three types of bodies in Box2D: static, kinematic, and dynamic. They are explained as follows:

- **Static bodies**: You can think of these as bodies that are fixed to the world. They don't move on their own, and they are not affected by gravity or collisions. Keep in mind that static bodies are able to collide with other objects, but they will not move or change as a result of the collision, just as the other objects (if they're not static). We'll use static bodies as platforms in our game, for our player to stand on.

- **Kinematic bodies**: These bodies are not affected by collisions or gravity either, but they can move on their own. They can affect other objects during collisions, as statics bodies do. A simple moving platform hazard should probably be implemented using a kinematic body.
- **Dynamic bodies**: These are affected by gravity, collisions, and can move on their own. We'll be using these to make our character and enemies move and collide. You can always set these bodies to ignore some force effects.

Now that we know a little bit more about bodies, let's add the floor to our game.

1. In order to do this, append the following code to `World:initialize()`.
2. Here we create a static body using `addBody` from `MOAIBox2DWorld`. We then use the method `setTransform` to place it. We center it in the `X` axis and place it at the bottom of the screen in the `Y` axis.

```
self.floor = {}
self.floor.body =
self.world:addBody ( MOAIBox2DBody.STATIC )
self.floor.body:setTransform ( 0, -WORLD_RESOLUTION_Y/2 )
```

3. After this we need to define the shape of the object (called fixture in the context of Box2D). We can use multiple kinds of shapes, and even combine them to create objects that are more complex. In this example, we're creating a rectangular object that has a width equal to the screen size and height equal to 100. The way we define these rectangles follows the convention of the rectangles on images, with the center at `(0,0)` on model coordinates.

```
self.floor.fixture = self.floor.body:addRect (
-WORLD_RESOLUTION_X/2, -50,
WORLD_RESOLUTION_X/2, 50 )
```

4. Then we define that we don't want friction on the floor. This will stop our player from slowing down when running.

```
self.floor.fixture:setFriction ( 0 )
```

Now we need to initialize our world in `game.lua` in order to render something to the screen. Just copy the following code below the call to `loadBackground`:

```
PhysicsManager:initialize ( self.layers.walkBehind )
```

What we're doing here is using the `walkBehind` layer as a parameter for debugging purposes. If you don't want to see the objects that are created, just avoid passing the layer and you won't see them.

Now run the game and you should see a box in the bottom. That's our floor.

Character versus the world

Now we want to make the main character interact with our world.

We'll add some new methods to the `Character` module as follows:

1. First we create an object to store our body and shapes as follows:

```
function Character:initializePhysics ()
    self.physics = {}
```

2. Then we add a dynamic body to the physics simulation. This will be the body of our character as follows:

```
    self.physics.body =
    PhysicsManager.world:addBody ( MOAIBox2DBody.DYNAMIC )
```

3. Now we position the body using the same initial position that we used for our rendered character as follows:

```
    self.physics.body:setTransform (
    unpack(character_object.position) )
```

4. Now we add a shape. We'll use a square that's the same size as the sprite (64 x 64). This is obviously far from ideal, because collisions may seem strange in some cases (it will feel like there was a transparent box around him), but in order to keep it simple, we can live with that.

```
    self.physics.fixture = self.physics.body:addRect
( -32, -32, 32, 32 )
```

5. In the preceding image, we see the character and a platform with the Box2D debug on them. We can see that the player should be falling, but it doesn't because its box is too big.

> You may want to create a better shape in your final game, in order to have better collision detection, but keep in mind that if the collision shape is more complex, it'll need more calculations when colliding. Always try to minimize the amount of vertices and edges of your fixtures.

6. The last step is to bind our `prop` with our `body`. To do so, we'll use the `setParent` method, which tells the Action Tree that the physics body affects the `prop` when moving. This is a great example demonstrating the power of the Action Tree.

```
    self.prop:setParent ( self.physics.body )
end
```

If you hit play now, the character should be falling due to gravity.

Movement

Now let's get right into the important stuff. First we'll define a couple of methods on `Character`. Then we'll rebuild `InputManager` to handle key events instead of mouse events and integrate that with our game.

1. The first method we're going to use will make our character run. It will be called when we press the *A* or *D* keys, moving left or right respectively. We'll pass it two parameters, `direction` (`-1` for left, `1` for right) and `keyDown`, which is a Boolean value that tells us if the player is pressing the key.
2. The first thing we do is use the `direction` parameter in combination with `setScl` (used to scale objects). The idea is that if you scale a sprite by a negative factor, it flips in the direction of the scaled component, so what we're doing here is telling our sprite to face left or right according to what key the player pressed.

```
function Character:run ( direction, keyDown )
    if keyDown then
    self.prop:setScl ( direction, 1 )
```

3. Then, we force the linear velocity by 100 in the specified direction and keep the velocity on the vertical axis, since we want to be able to jump and move while jumping:

```
    velX, velY = self.physics.body:getLinearVelocity ()
    self.physics.body:setLinearVelocity ( direction * 100,
    velY )
```

4. Then if we're not already running and we're not jumping, we start the run animation as following:

```
    if(self.currentAnimation ~= self:getAnimation ( 'run' ) )
     and not self.jumping then
         self:startAnimation('run')
     end
```

5. If this event was `keyUp` and we were not jumping, then stop moving. This is buggy, you should have a better logic here that keeps the state of the different keys, but to keep it simple we can live with it. And that's it, our `Character` can now run:

```
    else
    if not self.jumping then
        self:stopMoving ()
    end
    end
end
```

6. The two methods are really simple. If we move left, we call run with `-1` as the direction; if we move right, we call run with `1` as the direction. In either case, we pass the `keyDown` we receive. This `keyDown` will be obtained from the `InputManager` later.

```
function Character:moveLeft ( keyDown )
    self:run ( -1, keyDown )
end
function Character:moveRight( keyDown )
    self:run ( 1, keyDown )
end
```

7. What we do to stop moving is (after checking that we're not jumping) stop all the movement, setting the linear velocity to 0 in both directions and start the idle animation. This is pretty simple:

```
function Character:stopMoving ()
    if not self.jumping then
        self.physics.body:setLinearVelocity ( 0, 0 )
        self:startAnimation ( 'idle' )
    end
end
```

8. For the jump, we're going to do something different. Instead of forcing the linear velocity, we'll apply a force. We could do this in the same way, but just in order to illustrate how to apply forces, we'll use it here. Then we set a variable called jumping to true; this will allow us to know that we're jumping, and finally start the jump animation. This is easy as well.

```
function Character:jump ( keyDown )
    if keyDown and not self.jumping then
```

```
        self.physics.body:applyForce ( 0, 8000 )
        self.jumping = true
        self:startAnimation ( 'jump' )
    end
end
```

9. Finally, we'll implement the `stopJumping` method. We basically set jumping to false and stop moving as follows:

```
function Character:stopJumping ()
    self.jumping = false
    self:stopMoving ()
end
```

10. Our last method will be the one used to handle collisions with the floor. In order to use it, we need to add a line to `initializePhysics`.
11. What we're doing now is setting up a `callback` method that will be executed when Box2D detects a collision with the player's shape. The parameters are the method itself, and which phase of the collision it should be called in is defined as following:

```
self.physics.fixture:setCollisionHandler (
    onCollide,
    MOAIBox2DArbiter.BEGIN )
```

> To check all the collision phases, take a look at the `setCollisionHandler` definition at `http://getmoai.com/docs/class_m_o_a_i_box2_d_fixture.html#a693a608fc6645b170d067845dd1a9c20`.

Now we need to create the `callback` method:

12. What we do now is check that the shape that we're colliding with is the floor, if it is, then we stop jumping as follow:

```
function onCollide ( phase, fixtureA, fixtureB, arbiter )
    if fixtureB == PhysicsManager.floor.fixture then
        Character:stopJumping ()
    end
end
```

Keyboard input

We'll have to change a couple of things in order to start using the keyboard:

1. First of all, we need to remove our `processInput` method and the call to on `Game:start` in `game.lua`.
2. Then we need to create a method that will be in charge of managing the key input from `InputManager`.

   ```
   function Game:keyPressed ( key, down )
       if key == 'right' then Character:moveRight(down)end
       if key == 'left' then Character:moveLeft(down)end
       if key == 'up' then Character:jump(down)end
   end
   ```

   This method maps some key presses with the methods we created on `Character`. This should be enough to handle the movement of our character.

3. Now, we'll have to remove all the code (except the module definition) from `input_manager.lua` and rewrite it using the following code:

   ```
   function InputManager:initialize ()
       function onKeyboardEvent ( key, down )
           if key == 119 then key = 'up' end
           if key == 97 then key = 'left' end
           if key == 100 then key = 'right' end
           Game:keyPressed(key, down)
       end
   ```

   1. `onKeyboardEvent` will be our keyboard callback. What we do here is just use the key code of *A*, *D*, and *W*, replace them with a readable string, and pass them as parameters to `keyPressed`, the method we implemented previously.

      ```
          MOAIInputMgr.device.keyboard:setCallback
          ( onKeyboardEvent )
      end
      ```

   2. Now it's only a matter of setting the callback and we're done.

4. Try running the game now. You should be able to move the character around and jump using *A*, *D*, and *W*.

Creating the scene

We'll now create the necessary code to define a level. We won't be messing with sprites for it; instead we'll just use the default debug shapes from Box2D. We'll create the entire physical world, and you should now be able to assign sprites to it (it's the same as what we did with the player).

1. First of all, we need to remove all the code that relates to the floor definition in `physics_manager.lua`, since now we'll be creating the floor with the rest of the platforms.
2. Now, on `game.lua`, just below the `background_objects` definitions, we'll create a definition for the scene:

```
local scene_objects = {
    floor = {
        type = MOAIBox2DBody.STATIC,
        position = {0, -WORLD_RESOLUTION_Y/2},
        friction = 0,
        size = {2 * WORLD_RESOLUTION_X, 10}
    }
    platform1 = {
        type = MOAIBox2DBody.STATIC,
        position = {100, -50},
        friction = 0,
        size = {100, 20}
    }
}
```

3. As you may see in the given code, we're defining two objects, namely, a floor and a platform. To each object we define its body type, its position, and size. Note that the floor is now just another scene object.
4. We're now going to iterate through that table and create all the objects in one single piece of code:

```
function Game:loadScene()
    self.objects = {}
    for key, attr in pairs(scene_objects)do
```

    1. We start iterating through all the `scene_objects`:

```
local body = PhysicsManager.world:addBody(attr.type)
```

    2. Here we create the Box2D body, using the type in the definition:

```
body:setTransform ( unpack(attr.position) );
width, height = unpack ( attr.size );
local fixture = body:addRect (
-width/2, -height/2, width/2, height/2 )
```

3. Then we set the size and position.

```
        fixture:setFriction ( attr.friction )
```

4. And finally the friction.

```
        self.objects[key] = { body = body, fixture = fixture }
        end
    end
```

5. Then we store the object under the defined name and we're done.

If you wish to add more stuff to the definitions (such as the vertices for a polygon shape, or something like that), it should be only a matter of deciding how to represent that in the definition and adding the necessary method calls to meet that definition inside `loadScene`.

Currently, only eight vertices are supported. You can modify Moai SDK's source code to add more.

1. Below the `PhysicsManager` initialization on `Game:initialize`, you should add a call to `loadScene`.

```
self:loadScene ()
```

2. There is one more thing that we need to do now and that's fix our `collision` method that was using the hardcoded floor.
   1. First we need another auxiliary method in `game.lua`:

```
function Game:belongsToScene ( fixture )
    for key, object in pairs (self.objects )do
        if object.fixture == fixture then
            return true
        end
    end
    return false
end
```

   2. In this method, we iterate through all the scene objects and see if the one passed as a parameter is one of them.
   3. Open `character.lua` and replace `onCollide` with the new version:

```
function onCollide ( phase, fixtureA, fixtureB, arbiter )
    if Game:belongsToScene ( fixtureB ) then
        Character:stopJumping ()
    end
end
```

3. What we do here is call the auxiliary method when a collision occurs in order to see if we're colliding with an object from the scene. In that case, we stop jumping. This will make a funny mechanic; run the game and you'll see why (Super Meat Boy? Anybody? Ok, when you collide with a vertical edge of a box, the player stops, allowing you to jump again!).

The way we created this scene is decoupled from the code that actually loads it. In the real world, you will want to be able to edit your levels externally and then load them in code.

The loading code will behave mostly in the same way as in this case, but you'll probably use another language (XML, JSON, and so on) to store your levels. Level Editors are your friends.

Do you know **Inkscape**? It's a vector graphics editor that uses SVG as the main format. SVG is XML! So it is highly parsable. By the way, it's open source and free. Another useful open source tool if you're working with tiles is **Tiled**, which will be really useful to create your maps, as it includes layers, tilemap support, and exports to Lua.

Exercises

Adding enemies to this scene should be familiar to you by now. You'll need to think about the artificial intelligence for the movement, but other than that, everything else has have already been seen. You should create it as a dynamic body, and modify the player collision handler in order to react properly (killing the player for example, or taking life from it).

Another great idea would be to implement Mario-style platforms where you can jump up through the bottom of a platform and then collide and stay on top. (You'll have to use Box2D's sensors and check the normal of collision in the collision handler. Then, based on the collision normal, only stop movement at the appropriate time).

Play around a bit with this, since playing is a big part of the learning process.

Summary

In this chapter, you might have felt like Newton. We did some experiments with Box2D in order to build the physics of our platformer prototype. We saw how to use the keyboard and make our character animate and move using the input from the user and collisions.

The next chapter will take care of showing some debug data to illustrate how to create HUDs.

10
Creating a HUD

The **Heads-Up Display** (**HUD**) is a key technique that game designers use in order to provide information to the user. It's basically a set of graphics or text that are detached from the world itself and that are used to tell the player things such as how much life or *mana* they have left, or what their score is.

We're going to create a simple HUD that will give us really useful information such as the direction in which the character is going and its position in world coordinates. This will be enough to illustrate how to create a HUD. You will want to display more useful information in your own game. So let's go!

The basics

Create a file called `hud.lua`, require it from `game.lua`, and let's start our module.

Our HUD implementation will be based on the idea that we don't want world coordinates to position objects that are not inside the world. Also, we're more used to designing layouts for UIs thinking about (0, 0) on the top-left corner and the `Y` axis growing downward.

So that's what we're going to do:

```
module ( "HUD", package.seeall )
function HUD:initialize ()
    self.viewport = MOAIViewport.new ()
```

It can be done as follows:

1. In order to use a different coordinate system, we'll create a new viewport.

   ```
   viewport:setSize ( SCREEN_RESOLUTION_X, SCREEN_RESOLUTION_Y )
   ```

2. This viewport will have the same size as the screen:

   ```
   viewport:setScale  (SCREEN_RESOLUTION_X, -SCREEN_RESOLUTION_Y )
   ```

3. It won't use world coordinates. Instead, it will use screen coordinates, but we'll invert the `Y` axis so that it grows downward.

```
viewport:setOffset ( -1, 1 )
```

4. Now here's something new. The `setOffset` method is used to move the viewport using the projection space. The projection space is a 2 x 2 rectangle with its `Y` axis pointing upwards. Passing `-1` as the `X` value to `setOffset` moves this projection space half a screen to the left, and passing `1` as the `Y` value moves it half a screen to the top, achieving our goal of having the (0, 0) coordinate on the top-left corner.

> You can see here that this is pretty useful; you could move the (0, 0) to wherever you want and the positions will be calculated based on that.

5. After this, we create a layer and load it into the render table like we're used to:

```
    self.layer = MOAILayer2D.new ()
    self.layer:setViewport ( self.viewport )
    -- Now we need to render the layer.
    local renderTable = MOAIRenderMgr.getRenderTable ()
    table.insert ( renderTable, self.layer )
    MOAIRenderMgr.setRenderTable ( renderTable )
end
```

6. Add a call to `HUD:initialize ()` at the bottom of `Game:initialize` and you should be good to go.

Now, we're ready to start creating our `HUD` elements.

Left or right, that's the question

Now we're going to display our information on the screen:

1. First of all, we need to define a font in `game.lua` inside the `resource_definitions` table.

   We provided one with the source code for this chapter, but you can use whatever font you like:

```
hudFont ={
    type = RESOURCE_TYPE_FONT,
    fileName = 'fonts/tuffy.ttf',
glyphs =
"abcdefghijklmnopqrstuvwxyzABCDEFGHI
```

```
    JKLMNOPQRSTUVWXYZ0123456789,.?!",
    fontSize = 26,
    dpi = 160
  }
```

This should be familiar, since we talked about it in *Chapter 6, Resource Manager*. Now we know `hudFont` will reference our font.

2. Let's go back to `hud.lua`, create a method called `initializeDebugHud`, and call it from `HUD:initialize`:

```
function HUD:initializeDebugHud ()
    self.font = MOAIFont.new ()
    self.font = ResourceManager:get ( "hudFont" )
```

We use the font resource that we've just created as the font for our textboxes:

```
    self.leftRightIndicator = self:newDebugTextBox ( 30, {10, 10,
100, 50} )
    self.positionIndicator = self:newDebugTextBox ( 30, {10, 50,
200, 100} )
end
```

3. Then we call an auxiliary method that we will create in a moment. It will receive the font size and a rectangle where the text will be placed. The rectangle consists of the coordinates for the upper-left and bottom-right corners of the box:

    1. First of all, we create `MOAITextBox`. This is the class that will be used to display text. It inherits from `MOAIProp`, so you'll be able to move it, insert it into a layer, and do everything else you can do with `MOAIProp`:

    ```
    function HUD:newDebugTextBox ( size, rectangle )
        local textBox = MOAITextBox.new ()
    ```

    2. We set the font we loaded previously:

    ```
        textBox:setFont ( self.font )
    ```

    3. Then we set the size using the `size` parameter:

    ```
        textBox:setTextSize ( size )
    ```

You can use multiple text styles in `MOAITextBox`; take a look at `setStyle` and `MOAITextStyle` in the documents.

4. We split the `rectangle` table into parameters using `unpack`:

```
    textBox:setRect ( unpack(rectangle) )
```

5. The rectangle defines a box on the screen to which the text is confined. Text won't render outside the rectangle defined using `setRect`.
6. We insert the textbox in the HUD's layer.

```
    layer:insertProp ( textBox )
```

7. And finally return the textbox, so it can be referenced later.

```
    return textBox
end
```

Now we have our `HUD` layer populated with the textboxes, but there is one more thing we need in order to finish our goal.

Updating information

We need to write something into those textboxes. What we'll do in this case is call an `update` method in `HUD` periodically, so that it can refresh the data and print it on the screen:

```
function HUD:update ()
    local x, y = Character.prop:getScl ()
```

1. We get the scale that we used to set the character's direction, and store it in a local variable `x`:

```
    if x > 0 then
        self.leftRightIndicator:setString ( "Left" )
    else
        self.leftRightIndicator:setString ( "Right" )
    end
```

2. If you remember what we did before, this should be pretty clear. In order to turn right we scaled the character's `prop` to `-1` in the `x` axis, and to `1` to turn the character to the left. So that's what we're doing here. If the `x` value of the scale is positive, then we're facing left. Otherwise, if it is `-1`, we're facing right. Then, depending on the case, we use `MOAITextBox:setString` to update the string with the correct direction.

```
    x, y = Character.physics.body:getPosition ()
    self.positionIndicator:setString ("(" ..
    math.floor(x) .. ", " ..
    math.floor(y) .. ")")
end
```

3. Finally we update the textbox; we get the position from the physics body. Using the `..` operator to concatenate strings, we generate the correct string formatted as `(x, y)`. We then use `math.floor` to round down the position, since it's a decimal number.

The only thing we need to do now is add a call to `HUD:update ()` on the `while` loop inside `Game:start` and we should see both debug strings on the screen.

If you are uncertain about the size and layout of the boxes, you can use `MOAIDebugLines.setStyle(MOAIDebugLines.TEXT_BOX)` in order to debug them. This will display a line around each textbox.

Summary

In this chapter we learned the basics about how to implement a `HUD` with Moai SDK. We dug deeper into the `MOAIViewport` configuration in order to create a special viewport that used a different coordinate system than the one we've been using so far. We also displayed text for the first time using `MOAITextBox`.

In the next chapter, we'll dive into another important aspect of games: sound and music.

11
Let the Right Music In!

Sound and music are always left for the final stages. It's so embarrassing that the same has happened with this book. If you're serious about the gaming experience you're creating, take music and sound into account from the beginning. Even if your game is not audio-centric, you should think of music and audio as feedback systems and treat them with the same importance as graphics.

In this chapter we'll learn how to play audio files inside your game using the built-in Untz sound system. We'll add background music and sound effects.

Audio manager

In order to add some music and effects to our game we'll create a module called `AudioManager` that will take care of loading sounds and playing them.

1. First of all, we need to create a file called `audio_manager.lua`. This file will be in charge of managing all our sound needs. In this case we will use Untz, but you could easily modify it to use FMOD as follows:

```
module ( "AudioManager", package.seeall )
local audio_definitions = {
  backgroundMusic = {
    type = RESOURCE_TYPE_SOUND,
    fileName = 'sounds/music.mp3',
    loop = true,
    volume = 1
  },
  jump = {
    type = RESOURCE_TYPE_SOUND,
    fileName = 'sounds/jump.wav',
    loop = false,
    volume = 1
  }
}
```

2. In the same way we defined the assets for our game, we will define all the sounds using the resource definition for the audio files. We saw this earlier in the *Chapter 6*, *Resource Manager*. If you don't understand what we're doing here, I suggest you go back and read about sound definitions in *Chapter 6*. The `sounds` table will act as a cache for our sounds. It will be as follows:

```
sounds = {}
```

3. On the initialization method, the first thing we do is load all the definitions we added. We do this by calling the `setDefinitions` method from `ResourceDefinitions` as follows:

```
function AudioManager:initialize ()
    ResourceDefinitions:setDefinitions ( audio_definitions )
```

4. Then, since we're using Untz, we need to initialize it as follows:

```
    MOAIUntzSystem.initialize ()
end
```

> You can call `MOAIUntzSystem.initialize` with two optional parameters: the sampling rate (which can be changed to match your audio files) and the number of frames per buffer.

5. In order to play our sounds, we'll define a method that loads them in the local `sounds` table as follows:

```
function AudioManager:get ( name )
    local audio = self.sounds[name]

    if not audio then
        audio = ResourceManager:get ( name )
        self.sounds[name] = audio
    end

    return audio
end
```

This method checks if the audio has been previously loaded; if it was not loaded, it loads and returns it. We split this from the `play` method that we'll write now because we may want to preload the sounds before playing them.

6. The next step is to create the `play` method:

```
function AudioManager:play ( name, loop )
    local audio = AudioManager:get ( name )
```

1. Here we get the actual audio. If it was preloaded, it can be played without any loading delays.

```
    if loop ~= nil then
        audio:setLooping ( loop )
    end
```

2. Here we can use the `loop` parameter to override the default `loop` attribute.

```
    audio:play ()
end
```

3. And finally, we play it.

It's worth mentioning that playing a sound multiple times will cause the previous play to cut off.

7. We'll now write a function to stop the audio:

```
function AudioManager:stop ( name )
    local audio = AudioManager:get ( name )
    audio:stop ()
end
```

8. We need to append this file in `main.lua` just above the required `game.lua`.

Background music

Now let's play some background music by following the given steps:

1. At the bottom of `Game:initialize` you should add the following method calls:

```
    AudioManager:initialize ()
```

2. We initialize the audio using `AudioManager:initialize` (the method we created earlier in this chapter):

```
    AudioManager:play ( 'backgroundMusic' )
```

3. And then we play the background music. Remember that we defined it in the `audio_definitions` table inside `audio_manager.lua`.

That should be it. Run the game and you should be listening to an amazing song by Milhouse Palacios.

Sound effects

We will add a sound that will be played when the character jumps.

Open up `character.lua` and add a call to `AudioManager:play` inside `Character:jump`:

```
function Character:jump ( keyDown )
    if keyDown and not self.jumping then
        AudioManager:play ( 'jump' )
        self.physics.body:applyForce ( 0, 8000 )
        self.jumping = true
        self:startAnimation ( 'jump' )
    end
end
```

That's it. Run it and listen to your jump.

Summary

In this chapter we created a new module that takes care of the sound. We added some background music and a jump sound effect.

We've covered a lot already, and you're ready to create a complete game with what we saw. The next chapter will guide you through deploying our *Concentration* game to iOS.

12
iOS Deployment

Now that we've covered a lot of what is needed to make a game, we can start talking about deploying your game to specific devices. In this chapter we'll modify the default host project for iOS in order to get our game working on it. For this, we'll make a copy of the sample host project that comes with Moai SDK and go through all the things you need to get your own game running.

Xcode project

First of all, if you dream about your game making it to the iTunes Store, you're going to have to get the latest Xcode and iOS SDK from Apple. You get access to them by registering to Apple's developer website (`https://developer.apple.com/`).

If you don't have any experience using Xcode, it's suggested that you read a quick tutorial, since that's out of the scope of this book. A few basic things you should know about Xcode are how to handle source files, how to manage targets, and some basic Objective-C in order to modify the host to fit your game.

Go ahead and open the Xcode project that is provided with the Moai SDK download. It's located at `hosts/ios/moai.xcodeproject`. If you use the project from that location, it's already configured to use the Moai SDK static libraries in `lib/ios`, but if you move it, you'll need to fix the corresponding paths inside the project configuration. We'll see how to do this soon.

The host

All the code for the iOS-specific host can be found inside the `Classes` folder. If you take a close look at them, you'll easily understand what they do. Basically they set up an OpenGL view, register all input methods to AKU, and configure other services that you may find handy (or that you can remove if you don't want them).

There is another folder named `Resources` (that contains icons), which is where the compiled source will go (inside another folder called `build`).

You'll see that, by default, the `moai-target` file and the `lua` folder are displayed in red. The reason for that is that during the build, a script runs and reads the content of `moai-target` in `Resources/build` to identify all the Lua source code for our game. After finding it, the script copies every directory referenced by `moai-target` into the `lua` folder (the files also get marked as read-only to remind you that they should not be edited, since they'll be deleted during the next build). The files inside the `lua` folder are included in the application bundle, so that they can be accessed when packing it for deployment. Take a look at `MoaiAppDelegate.mm`, where the code for calling `main.lua` to start your game is located. If you want to use a different entry point for `main.lua`, you can do it there.

In the `moai-target` file, you should include all the code and asset sources for your game, one per line. For example:

```
../../myGame/src
../../myGameAssets/
```

If there are special dependencies on a module that you're installing (check `samples/iphone/app-apsalar` for an example), you can use a file called `moai-target-ext` to define them. The script will check for this file inside each folder that you define in `moai-target` and recursively copy all the contents in it as well.

This is crucial for getting things working with the host that is provided.

You should also take a look at the `Frameworks` folder, which has all the static libraries and iOS frameworks needed to build our host.

Running an example

Before starting to modify your game code, you should try to run the sample project. It already points to one of Moai SDK's samples, so it should be as easy as checking that the device in the Simulator and hitting **Build and Debug**.

If you see Moai SDK's logo rotating, you're good to go. If not, check if your Xcode configuration is OK, or join Moai SDK's forum and ask for help, detailing the errors you get.

Running on an actual device requires you to understand (or at least hack into—who can understand it?) how to handle and set up Apple's provisioning profiles for development. This is carefully explained in the documentation that you can find at Apple's developer portal (`https://developer.apple.com/devcenter/ios/index.action`).

Setting up our own project

Since the Xcode project sample that comes with Moai SDK includes a bunch of things that we may not need, it's recommended to do a clean up. We'll start by copying the whole project somewhere in our hard drive. If you have been following all the chapters and have extracted all the code, a good place to put our project is in the same folder as the code samples, so that we can point at them and have our *Concentration* game available on iOS. You can use the source code of *Chapter 7*, *Concentration Gameplay*.

Either way, if you downloaded the book's source code, you'll find the iOS project and the *Concentration* gameplay implementation in the code for this chapter.

Now that we have both the Xcode project and the *Concentration* game in place, double-click on `moai.xcodeproj` in order to open Xcode and start configuring it.

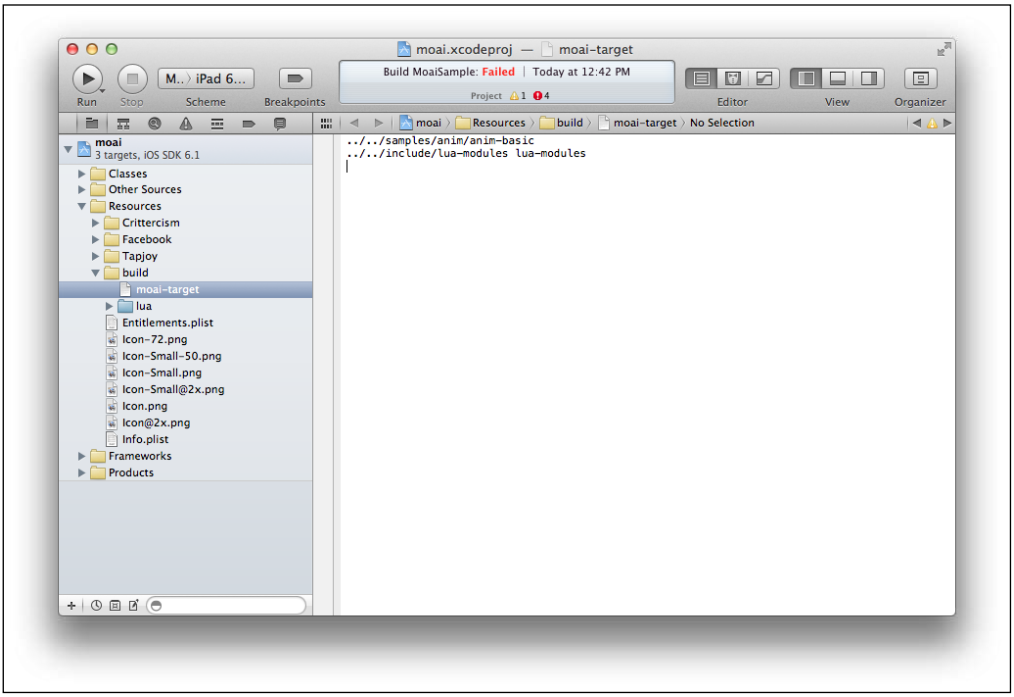
It may take a while to load. Xcode performs file indexing the first time you open a project. Don't worry, that won't happen on subsequent loads (well, except when it decides to index randomly).

If you run it now, it will probably fail. Learn to take your time to read and understand the errors; that's the only way to solve them.

Now, let's fix the project step by step.

Pointing to the correct source code

The default project points to one of Moai SDK's examples using a relative path. Since we moved the project, that path is now broken. We need to make everything point to our own code.



In order to do this, expand the **Resources** folder, then the **build** folder, and finally click on **moai-target**.

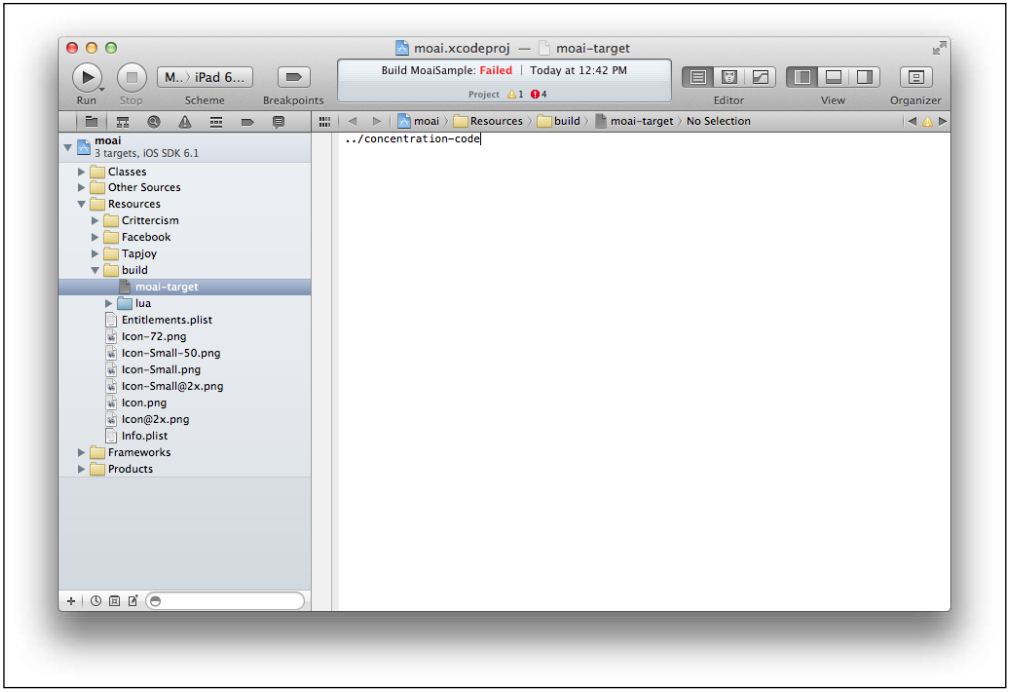
The content of `moai-target` should be as follows:

```
../../../samples/anim/anim-basic
../../../include/lua-modules lua-modules
```

`moai-target` is pointing to the `anim-basic` sample using relative paths. So, in order to run our own game, we need it to point to the correct location.

In this example, we've moved the game's code to a folder called `concentration-code`, so that's where we will point to. The final `moai-target` should look as follows:

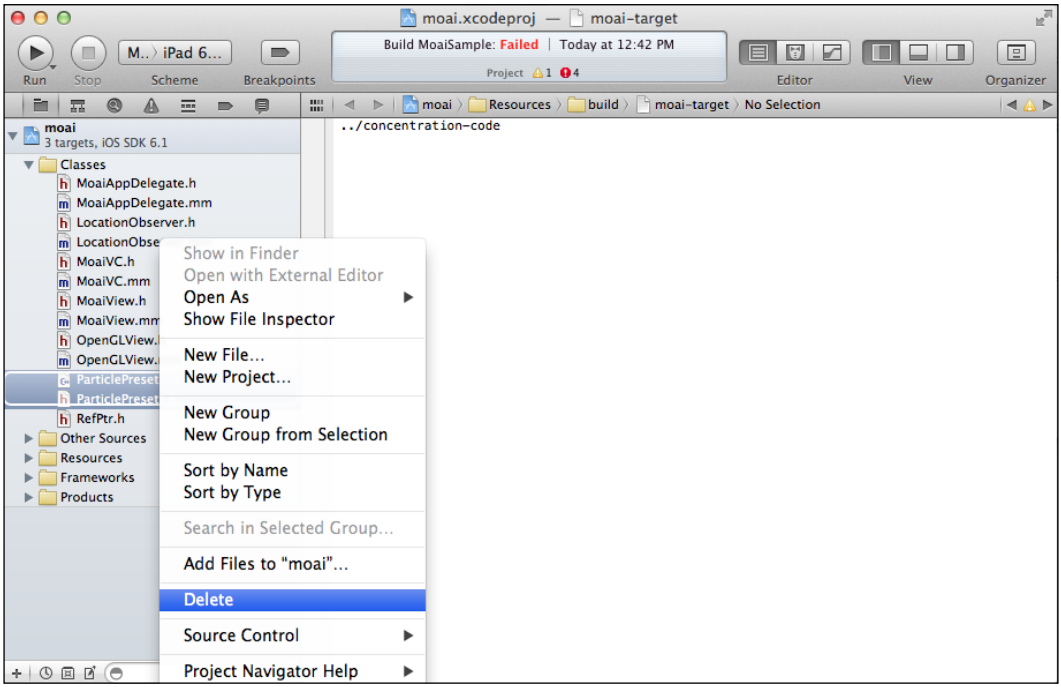
```
../concentration-code
```



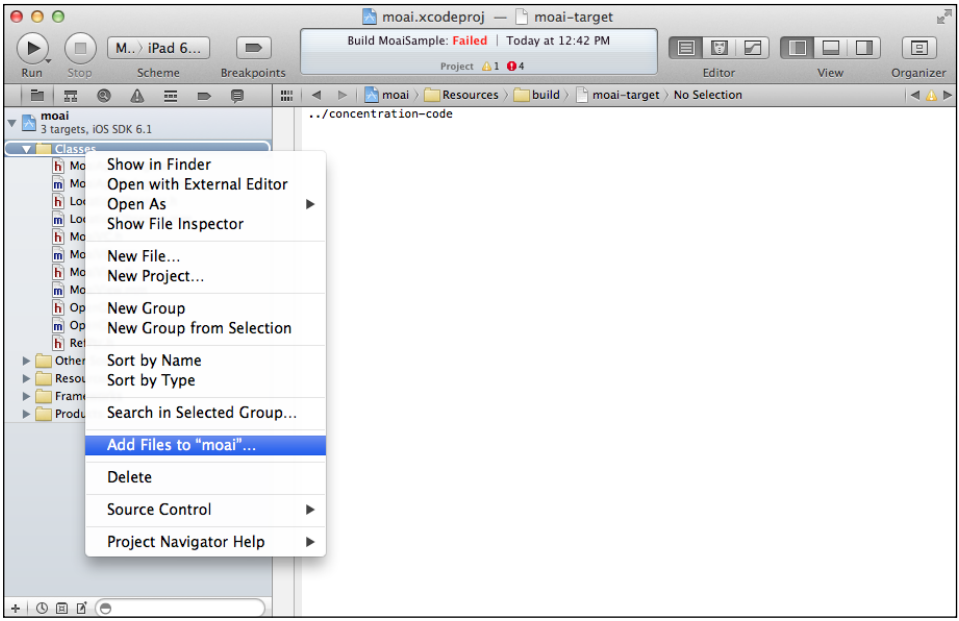
Fixing source paths

Since we moved the Xcode project from its original location, now we need to fix relative paths around the it.

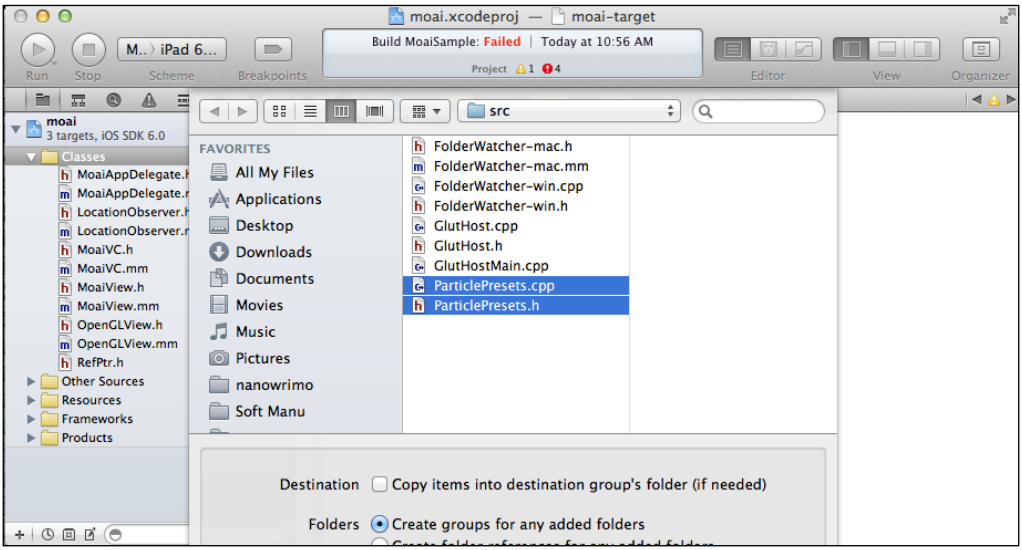
1. Collapse the **Resources** folder and expand **classes**. You'll see that **ParticlePresets.cpp** and **ParticlePresets.h** are in red. They can be found in `moai-sdk/hosts/src/` (`moai-sdk` being the folder where you installed Moai SDK in the initial chapters).



2. Select **ParticlePresets.cpp** and **ParticlePresets.h** and remove them.



3. Now right-click on the **Classes** folder and choose **Add Files to "moai"**. Go to `moai-sdk/hosts/src/` and pick both **ParticlePresets.cpp** and **ParticlePresets.h**. After you accept, they should be in the folder and should not be displayed in red.



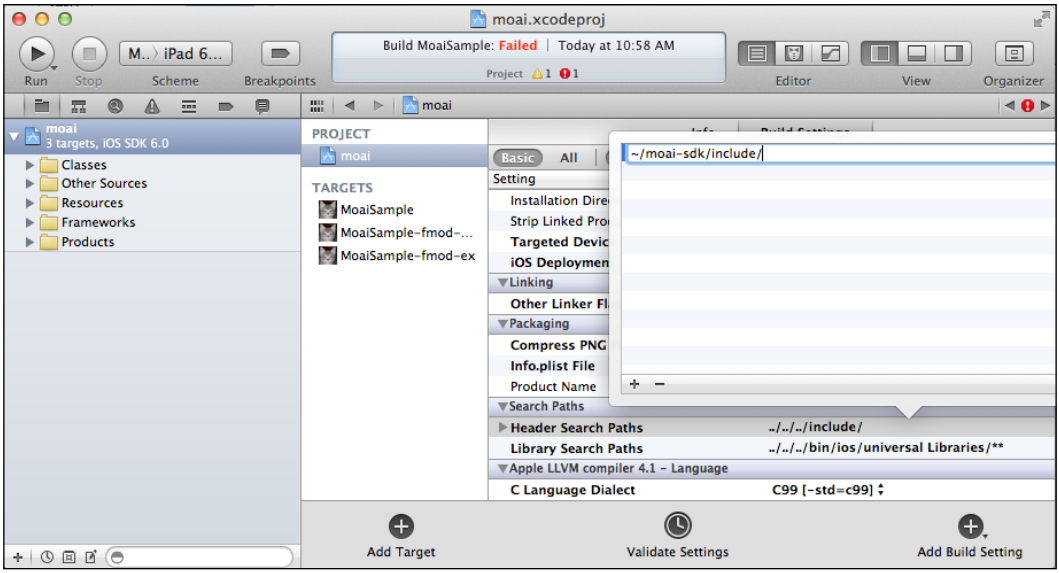
Fixing include paths

The next thing we need to do is fix the relative include paths.

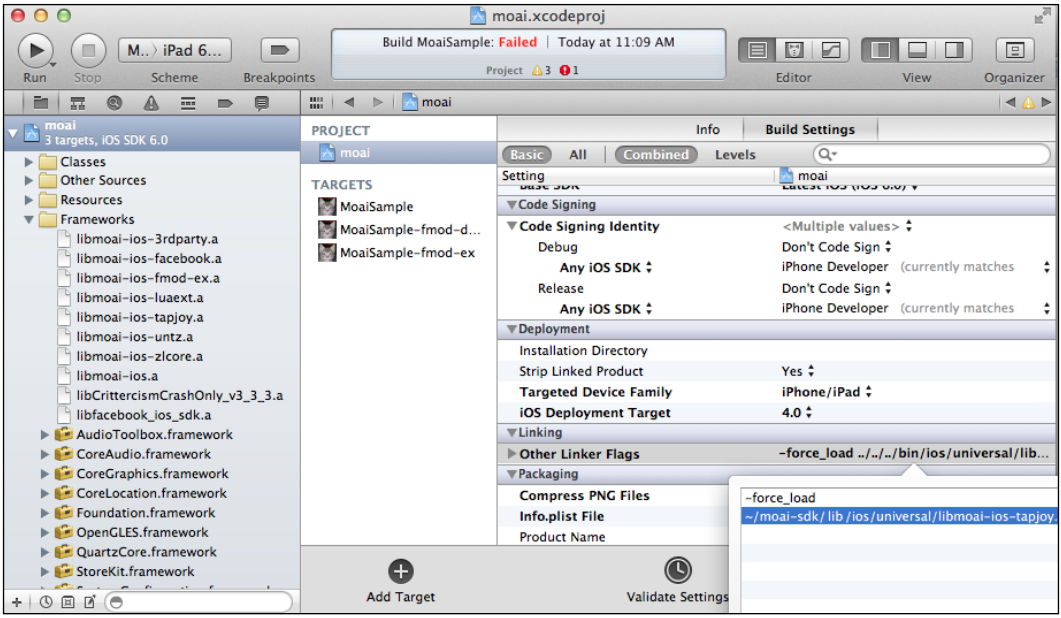
To do this, click on the top of the folder tree, where it says **moai** and the icon for the Xcode application is displayed. Then, under the **PROJECT** section (not **TARGETS**), select **moai**.

There are a couple of things to fix here:

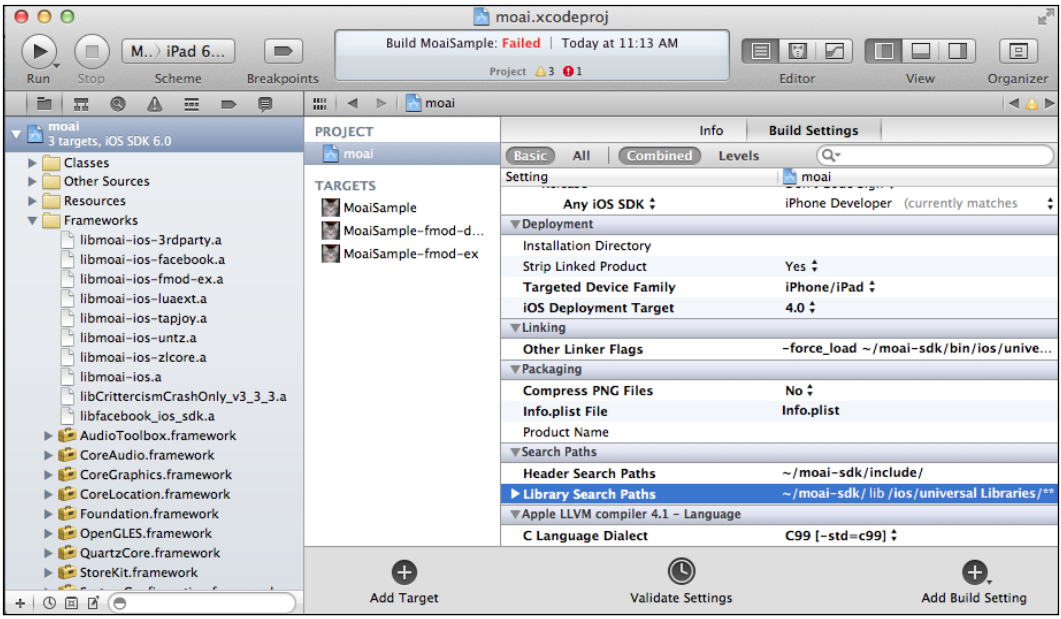
1. Search for the setting named **Header search paths**; it should read **../../include/**. If you double-click on it, a popup will be displayed and you can then double-click the include path and change it to your own include path, which is located at `moai-sdk/include/`.



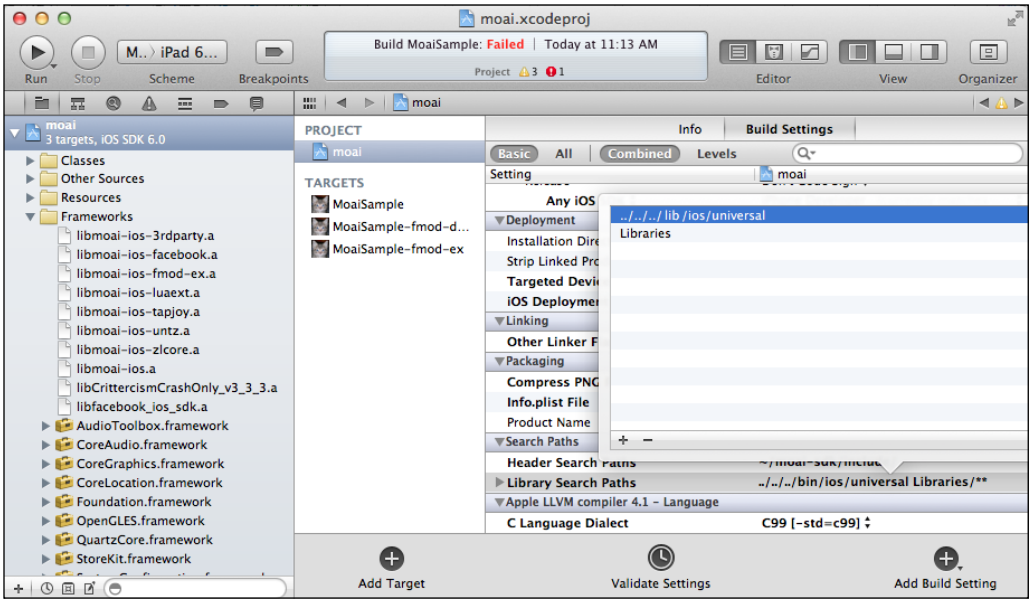
2. Search for **Other Linker Flags**, and change **../../../lib/ios/universal/libmoai-ios-tapjoy.a** to the correct location (`moai-sdk/lib/ios/universal/libmoai-ios-tapjoy.a`). If Xcode still can't find this file, try using an absolute path (in my case it is `/Users/nictuku/moai-sdk/lib/ios/universal/libmoai-ios-tapjoy.a`).



3. Search for **Library Search Paths** and change **../../../lib/ios/universal** to `moai-sdk/lib/ios/universal`.

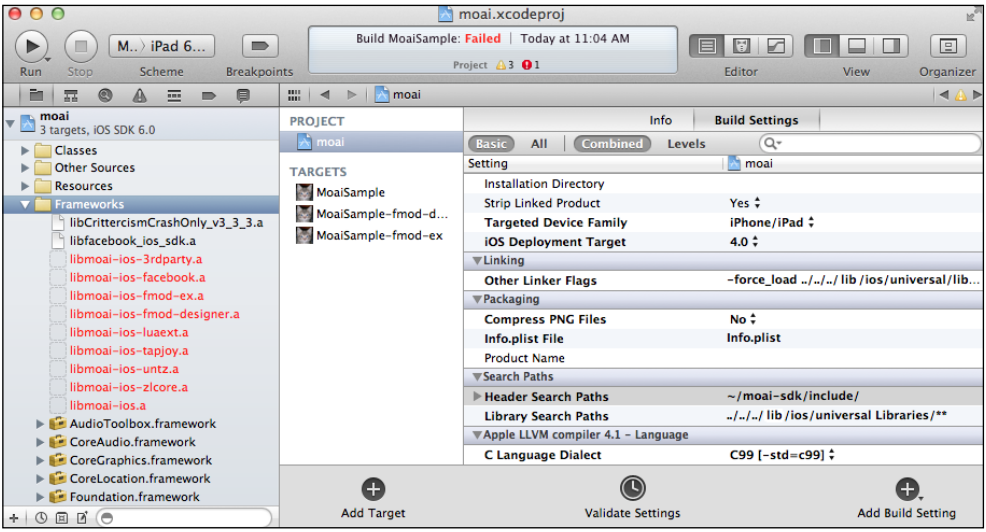


4. Now, click on **MoaiSample** under the **TARGETS** section. Double-click on **Library Search Paths** and replace the remaining relative paths in the same way we changed the previous ones.



Fixing linked libraries

The next thing we need to fix is the location for all the linked libraries.

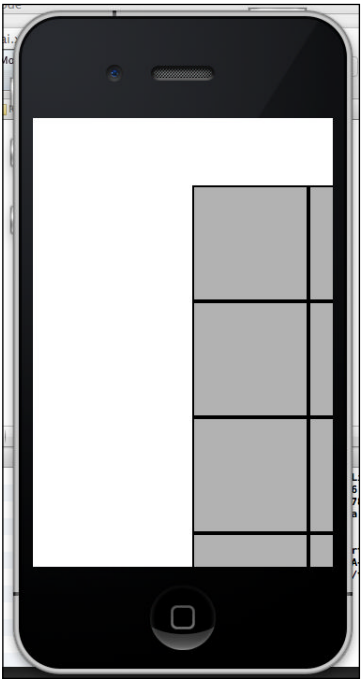


Expand the **Frameworks** folder and identify all the libraries that show up in red.

You can find them in `moai-sdk/bin/ios/universal`. Remove the ones in red, and add all the files in that path, as we did with the `ParticlePresets` files.

You should end up with the same files in the directory but they should not be displayed in red anymore.

Now, if you hit **Run**, the build should succeed and you should see our *Concentration* game on the iOS Simulator.



But wait! It's not being displayed correctly, and touches are not detected! That's because of the way we configured our project locally. So let's try to fix that.

Going multi-platform

The first thing we need to think about is how we are going to deal with different resolutions in our project. You could do a lot of fancy stuff in this regard, such as changing the UI given different sizes, or loading HD images if the resolution is higher than or equal to 1024 x 768 (or whatever you choose!), and so on. In this example we're going to keep it simple; we will just stretch everything so that it fits on the screen.

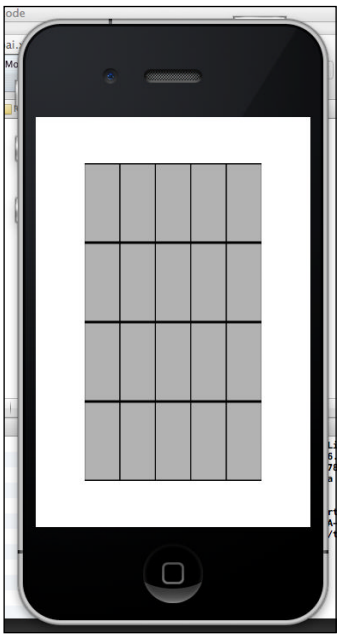
Open the `Concentration` project and change the following lines:

```
SCREEN_RESOLUTION_X = 2 * WORLD_RESOLUTION_X
SCREEN_RESOLUTION_Y = 2 * WORLD_RESOLUTION_Y
```

To the following:

```
SCREEN_RESOLUTION_X = MOAIEnvironment.horizontalResolution
SCREEN_RESOLUTION_Y = MOAIEnvironment.verticalResolution
```

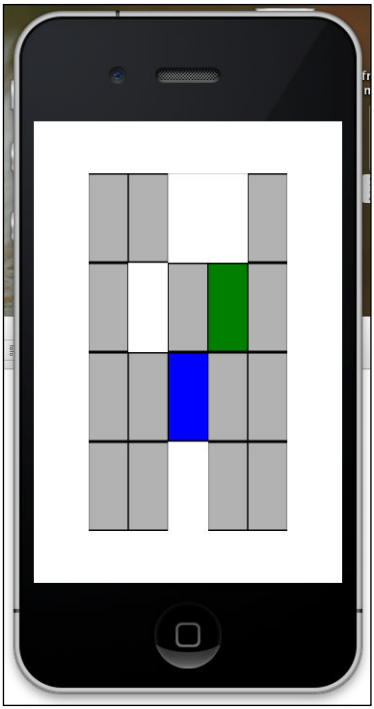
If you run the project now, it should show the stretched version of the game, but it still does not react to touches.



In order to fix this, edit `game.lua` and modify `processInput` to use `InputManager:getTouch` instead of `InputManager:position`.

Where it says `x, y = InputManager:position ()`, you should replace that with `x, y = InputManager:getTouch ()`.

Now if you run your project, you'll be able to play our game.



But it looks stretched. You can learn how to make it work in landscape mode at Moai's wiki page (`http://getmoai.com/wiki/index.php?title=Setup_landscape_in_Moai_Games_For_iOS_Devices`).

Running on the device

After all the steps we followed, the game should be able to run on a device. It should just be a matter of setting the correct **code sign** identities and changing the target device.

It would be great for you to try this now and ask for help from the Apple community or the Moai forum if you're lost, since setting up an Apple Developer account and certificates exceed the scope of this book.

Summary

In this chapter, we learned how to modify the Xcode project for the iOS host in order to get it working with our own game. We fixed the relative paths that broke when we moved the project and set the resolution correctly.

We know iOS is not the only platform around there. We'll talk about other platforms in the next chapter!

13 Deployment to Other Platforms

Continuing the talk about multiplatform development, we should spend some time analyzing what other platforms are supported by Moai SDK.

Moai SDK ships with example hosts for different platforms. It's recommended that you read and understand the code for the platform's hosts, so you are able to modify them. If you wish to add your own fancy advertising library, or integration with the next generation social network, you'll need to modify the host.

Let's take a look at the hosts that come in the Moai SDK release.

Windows

In order to build for Windows you need to use one of the Visual Studio projects. You'll find both of them inside `moai-sdk/hosts/folder` (`vs2010` and `vs2008`).

Open up the correct solution for your Visual Studio version. Inside, you'll find all the code you need for the **GlutHost**, which should run smoothly on windows. Moai SDK already ships with a compiled version of this host in `moai-sdk/bin/win32/`.

Mac OS X

The host for Mac OS X can be compiled using the Xcode project from `moai-sdk/hosts/xcode/osx`.

The codebase for this host is basically the same as for the Windows one; it uses the GlutHost program with some considerations to make it work on Mac OS X.

Android

The Android hosts are generated using a shell script located in `moai-sdk/hosts/android/run-host.sh` (or `.bat`). Be sure to have Apache Ant installed and all the Android stuff as well. Take a look at the wiki page for more information regarding building your game in Android (`http://getmoai.com/wiki/index.php?title=Building_Moai_Games_For_Android_Devices`).

Compiling for Android could be a whole chapter by itself. Besides the mentioned link in Moai SDK wiki, you'll also need some knowledge of Android development, so it's recommended that you head over and read Android's documents before trying to build the host (`http://developer.android.com/develop/index.html`).

Google Chrome (native client)

Native client (`https://developers.google.com/native-client/`) is a new way to introduce code that runs natively in the computer and that can be downloaded from your Chrome browser. Moai SDK comes with a host built for it, which can be found in `moai-sdk/hosts/chrome`.

You'll need to change the `manifest.json` file and put all your code in the same folder.

Detailed instructions on how to do this are located at Moai's wiki link (`http://getmoai.com/wiki/index.php?title=Building_Moai_Games_for_Google_Chrome`).

Linux

At the time of writing the book, Linux is not available as a target platform in the Moai SDK release, but Moai works on it building from the source code.

There are two main ways to do this right now:

- Using the Linux branch from the Moai SDK's GitHub repository as explained at `http://franciscotufro.com/2013/01/compiling-moai-on-linux/`
- Using the spacepluk's branch as explained at `https://github.com/spacepluk/moai-dev`

The second method is preferred for now since it has Untz support, but official Linux support is on the way, so double check the latest Moai SDK release to verify that Linux is not yet officially supported.

Insert the next platform here

We mentioned this before, but Moai SDK has been developed in a way that it should be easy to port to whatever platform that supports C++ and OpenGL. If you are able to get it working for a new platform, you can share your experience and send a pull request to `https://github.com/moai/moai-dev`. We will all be grateful!

Summary

In this chapter we summarized the different platforms that are currently supported by Moai SDK. We pointed to where to get information about building a host for those platforms.

What do you say? Are you ready to create the next big game? I bet you are! Go ahead, create the best game ever and don't forget to tell the community about it!

Join the forums at `http://getmoai.com/forums/` and be a part of this amazing game-development community. We're waiting for you.

Thanks for reading the book and supporting open source technologies. Hope to see you around soon!

Index

A

Action Tree 18
addAnimation() method 76
AKU 20
Android
  in Moai SDK release 114
animations 71, 72
Apple
  URL, for developer website 99
ASSETS_PATH constant 39
audio files
  playing 95-97
audio manager 95
audio_manager.lua file 95

B

background
  inserting, in screen 63, 64
  splitting, in images 66
background music
  playing, steps 97
blend modes 34
Box2D
  body types 78
  world, creating 77, 78
Box2D body, types
  dynamic body 79
  kinematic body 79
  static body 78

C

C++ 20, 21
callback method
  creating 83
camera
  using 65
character
  animations, performing 71-76
  creating 69, 71
character module
  methods, adding 80, 81
code sign identities 111
concentration game. *See* **game**
coordinate system
  about 32
  model coordinates 33, 34
  window/screen coordinates 33
  world coordinates 33
coroutine.yield() method 57

D

decks
  about 29
  MOAIGfxQuad2D deck 29
  MOAITileDeck2D deck 29
dynamic body 79

E

EMPTY_TILE constant 61

F

final method 38
FMOD 46
fonts 45

G

game
designing 23, 24
floor, adding 79
image, manipulating 29
MOAISim.openWindow class method 25
optimizing 8
project, setting up 25
window opening, Moai SDK used 25
world resolution versus screen/window resolution 26, 28
game design document. *See* **GDD**
Game:initialize() method 55
gameplay implementation
about 50-55
auxiliary methods 61, 62
cell-choosing method 58, 59
initialization 55, 56
input handling 54
input processing 56-58
tiles, removing 61
tiles, resetting 60, 61
tile swapping 60
Game:start() method 51
GDD 7
getter method 38, 40
GlutHost 113
GNU/Linux
Moai SDK, installing 14
Google chrome
in Moai SDK release 114
grid
displaying 49

H

Heads-Up Display. *See* **HUD**
hosts 20
hosts, Moai SDK release
Android 114
Google chrome 114
Linux 114, 115
Mac OS X 113
Windows 113
host, Xcode project 99, 100
HUD
about 89
basics 89, 90
creating 89, 90
information, displaying on screen 90-92
information, updating 92, 93
HUD:initialize() method 90

I

images
about 30
background, splitting 66
displaying, on layer 67-69
loading 42, 43
include paths
fixing 106-108
initializeDebugHud method 91
initialize() function 51, 64
initializeTiles() method 52
Inkscape 87
InputManager
rebuilding 81, 82
Input Queue 18
iOS Deployment
Xcode project 99
isDown() method 54

K

key 74
keyboard
input 84
keyDown 82
kinematic body 79

L

layer
about 30
creating 30, 31
images, displaying on 67-69
setting up 67
linked libraries
fixing 108, 109
Linux
in Moai SDK release 114, 115

loader, resource manager
about 40, 41
fonts 45
images 42, 43
sounds 46
tiled images 44
loadFont method 45
loadGfxQuad2D method 43
loadImage method 42
load method 40
Lua 20, 21, 39

M

Mac OS X
in Moai SDK release 113
Moai SDK, installing 13
MOAIAnim 71
MOAIAnimCurve 71
MOAIBox2DWorld object 77
MOAICamera2D class 65
MOAICoroutine.blockOnAction() method 62
MOAIDeckRemapper 71, 72
MOAIEaseType 65
MOAIGfxQuad2D deck 29
MOAIGrid class 52
MOAILayer 30
MOAILayer2D method 34, 66
MOAIProp 30
MOAIProp2D.BLEND_ADD 34
MOAIProp2D.BLEND_MULTIPLY 35
MOAIProp2D.BLEND_NORMAL 34
MOAIProp2D method 34
Moai SDK
about 8, 12
installing, on GNU/Linux 14
installing, on Mac OS X 13
installing, on Windows 13
sample, running 14, 15
Moai SDK release
hosts 113-115
Moai SDK Runtime
data structures 17
MOAISim.openWindow class method 25
Moai's wiki
URL 12
MOAITileDeck2D deck 29
Moai update loop
steps 17
Moai update loop, steps
about 17
Action Tree 18
Input Queue 18
Node Graph 18, 19
MOAIViewport instance 27
model coordinates 33
moveLoc() method 65

N

Node Graph 18, 19

P

Parallax 66
processInput method 84
project
linked libraries, fixing 108, 109
Prop 52
props 29

R

render manager 19
render table 32
resetTiles() method 60
resource definitions
about 37, 38
final method 38
getter method 38
setter method 38
ResourceDefinitions 37
resource manager
about 39
getter method 40
loader 40, 41
ResourceManager 37

S

scene
creating 85-87
screen
background, inserting in 63, 64

SCREEN_RESOLUTION_X constant 27
SCREEN_RESOLUTION_Y constant 27
screen/window resolution
  versus world resolution 26-28
setBlendMode() method 34
setDefinitions() method 51, 96
setGrid() method 52
setLoc() method 32, 53
setOffset method 90
setParallax() method 66
setRenderTable() method 32
setRow() method 52
setter method 38
setTexture() method 30
setViewport() method 30
Shader Model 64
sound 95
sound effects
  adding, to game 98
sounds 46
source paths
  fixing 104, 105
static body 78
swapTile() method 60

T

tiled images 44
tilemaps 49
tiles
  removing 61
  resetting 60, 61
tileset
  creating 49, 50
tilesets 49
Tron-like lights 34

U

Untz 46
Untz sound system
  audio files, playing 95-97

W

Windows
  in Moai SDK release 113
  Moai SDK, installing 13
window/screen coordinates 33
world coordinates 33
world resolution
  versus screen/window resolution 26-28

X

Xcode project
  about 99
  going multi-platform 109, 110
  host 99, 100
  include paths, fixing 106-108
  pointing, to correct source code 102, 103
  setting up 101
  source paths, fixing 104, 105

Z

ZeroBrane Studio
  about 12
  URL, for installing 12

[PACKT] PUBLISHING open source community experience distilled

Thank you for buying

Developing Mobile Games with Moai SDK

About Packt Publishing

Packt, pronounced 'packed', published its first book "*Mastering phpMyAdmin for Effective MySQL Management*" in April 2004 and subsequently continued to specialize in publishing highly focused books on specific technologies and solutions.

Our books and publications share the experiences of your fellow IT professionals in adapting and customizing today's systems, applications, and frameworks. Our solution based books give you the knowledge and power to customize the software and technologies you're using to get the job done. Packt books are more specific and less general than the IT books you have seen in the past. Our unique business model allows us to bring you more focused information, giving you more of what you need to know, and less of what you don't.

Packt is a modern, yet unique publishing company, which focuses on producing quality, cutting-edge books for communities of developers, administrators, and newbies alike. For more information, please visit our website: `www.packtpub.com`.

About Packt Open Source

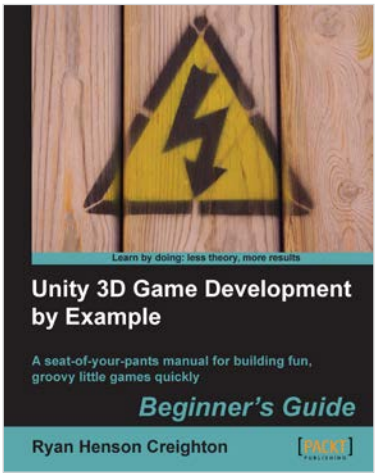
In 2010, Packt launched two new brands, Packt Open Source and Packt Enterprise, in order to continue its focus on specialization. This book is part of the Packt Open Source brand, home to books published on software built around Open Source licences, and offering information to anybody from advanced developers to budding web designers. The Open Source brand also runs Packt's Open Source Royalty Scheme, by which Packt gives a royalty to each Open Source project about whose software a book is sold.

Writing for Packt

We welcome all inquiries from people who are interested in authoring. Book proposals should be sent to author@packtpub.com. If your book idea is still at an early stage and you would like to discuss it first before writing a formal book proposal, contact us; one of our commissioning editors will get in touch with you.

We're not just looking for published authors; if you have strong technical skills but no writing experience, our experienced editors can help you develop a writing career, or simply get some additional reward for your expertise.

[PACKT] PUBLISHING open source community experience distilled



Unity 3D Game Development by Example Beginner's Guide

ISBN: 978-1-849690-54-6 Paperback: 384 pages

A seat-of-your-pants manual for building fun, groovy little games quickly

1. Build fun games using the free Unity 3D game engine even if you've never coded before
2. Learn how to "skin" projects to make totally different games from the same file – more games, less effort!
3. Deploy your games to the Internet so that your friends and family can play them

---

Unreal Development Kit Game Programming with UnrealScript: Beginner's Guide

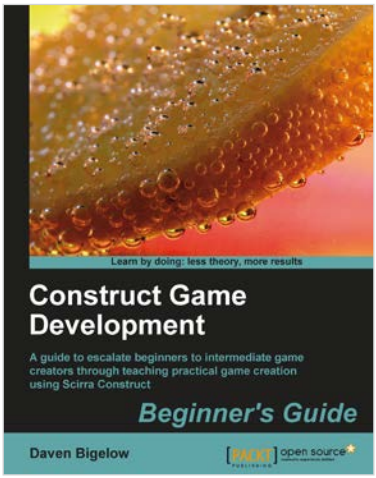
ISBN: 978-1-849691-92-5 Paperback: 466 pages

Create games beyond your imagination with the Unreal Development Kit

1. Dive into game programming with UnrealScript by creating a working example game.
2. Learn how the Unreal Development Kit is organized and how to quickly set up your own projects
3. Recognize and fix crashes and other errors that come up during a game's development

Please check **www.PacktPub.com** for information on our titles

[PACKT] open source
community experience distilled
PUBLISHING



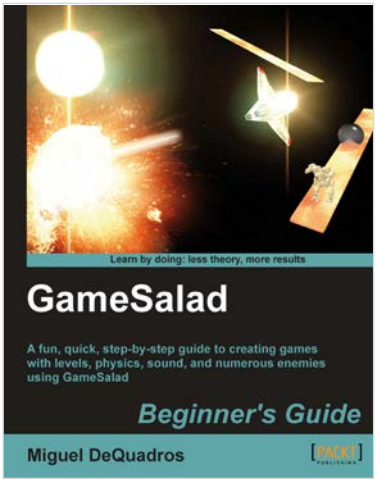
Construct Game Development Beginners Guide

ISBN: 978-1-849516-60-0 Paperback: 298 pages

A guide to escalate beginners to intermediate game creators through teaching practical game creation using Scirra Construct

1. Learn the skills necessary to make your own games through the creation of three very different sample games
2. Create animated sprites, use built-in physics and shadow engines of Construct Classic
3. A wealth of step-by-step instructions and images to lead the way

---



GameSalad Beginner's Guide

ISBN: 978-1-849692-20-5 Paperback: 308 pages

A fun, quick, step-by-step guide to creating games with levels, physics, sound, and numerous enemies using Gamesalad

1. Learn to build three games; Ball Drop, Space Defender, and Metal Mech with GameSalad
2. Complete these games with sound effects, use of physics, gravity, and collisions
3. Learn tips and tricks to make a game popular straight from the author's own experience.

Please check **www.PacktPub.com** for information on our titles